

**IMPLEMENTASI *HYBRID INTRUSION PREVENTION SYSTEM*
(IPS) MENGGUNAKAN SURICATA DAN DECISION TREE
UNTUK MENCEGAH *NETWORK RECONNAISSANCE* BERBASIS
TCP**

SKRIPSI

Untuk memenuhi sebagian persyaratan
memperoleh gelar Sarjana Komputer

Disusun oleh:

Virgananta Saputra

NIM: 225150201111028



PROGRAM STUDI TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026

PENGESAHAN

IMPLEMENTASI *HYBRID INTRUSION PREVENTION SYSTEM* (IPS) MENGGUNAKAN
SURICATA DAN DECISION TREE UNTUK MENCEGAH *NETWORK RECONNAISSANCE*
BERBASIS TCP

SKRIPSI

Diajukan untuk memenuhi sebagian persyaratan
memperoleh gelar Sarjana Komputer

Disusun oleh :

Virgananta Saputra

NIM: 225150201111028

Skripsi ini telah diuji dan dinyatakan lulus pada:

06 Mei 2026

Telah diperiksa dan disetujui oleh:

Dosen Pembimbing 1

Dosen Pembimbing 2

Mahendra Data, S.Kom., M.Kom., Ph.D.

NIK: 2015038611171001

Dr. Kasyful Amron, S.T., M.Sc.

NIP: 197508032003121003

Mengetahui

Ketua Departemen Teknik Informatika

Rekyan Regasari Mardi Putri, S.T., M.T., Ph.D.

NIK: 2011027704142001

PERNYATAAN ORISINALITAS

Saya menyatakan dengan sebenar-benarnya bahwa sepanjang pengetahuan saya, di dalam naskah skripsi ini tidak terdapat karya ilmiah yang pernah diajukan oleh orang lain untuk memperoleh gelar akademik di suatu perguruan tinggi, dan tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis disitasi dalam naskah ini dan disebutkan dalam daftar referensi.

Apabila ternyata di dalam naskah skripsi ini dapat dibuktikan terdapat unsur-unsur plagiasi, saya bersedia skripsi ini digugurkan dan gelar akademik yang telah saya peroleh (sarjana) dibatalkan, serta diproses sesuai dengan peraturan perundang-undangan yang berlaku (UU No. 20 Tahun 2003, Pasal 25 ayat 2 dan Pasal 70).

Malang, 12 Februari 2026



Virgananta Saputra

NIM: 225150201111028

PRAKATA

Segala puji dan syukur penulis panjatkan ke hadirat Allah SWT atas segala limpahan rahmat, taufik, dan hidayah-Nya sehingga penulis dapat menyelesaikan penyusunan skripsi yang berjudul “Implementasi *Hybrid Intrusion Prevention System* (IPS) Menggunakan Suricata dan Decision Tree Untuk Mencegah *Network Reconnaissance* Berbasis TCP”.

Skripsi ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Komputer pada Program Studi Teknik Informatika, Fakultas Ilmu Komputer, Universitas Brawijaya. Penyusunan skripsi ini tidak lepas dari bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis menyampaikan rasa terima kasih yang sebesar-besarnya kepada:

1. Diri penulis sendiri karena telah berusaha melakukan segala cara agar skripsi ini dapat selesai dengan sebaik baiknya,
2. Bapak Mahendra Data, S.Kom., M.Kom., Ph.D. dan Bapak Dr. Kasyful Amron, S.T., M.Sc. selaku dosen pembimbing yang telah sabar, banyak membantu dan berdiskusi dalam proses pengerjaan skripsi,
3. Bapak Ir. Lutfi Fanani, S.Kom., M.T., M.Sc. Selaku Dosen Penasehat Akademik yang selalu mengarahkan dan memberikan nasihat kepada penulis selama menempuh masa studi di Fakultas Ilmu Komputer, Universitas Brawijaya,
4. Bapak Bayu Priyambadha, S.Kom., M.Kom., Ph.D. Selaku Ketua Program Studi Teknik Informatika,
5. Ibu Rekyan Regasari Mardi Putri, S.T., M.T., Ph.D. Selaku Ketua Departemen Teknik Informatika,
6. Bapak Ir. Tri Astoto Kurniawan, S.T., M.T., Ph.D., IPM. Selaku Dekan Fakultas Ilmu Komputer,
7. Seluruh dosen dan staf akademik di Fakultas Ilmu Komputer Universitas Brawijaya yang telah memberikan bekal ilmu dan fasilitas selama masa studi,
8. Bapak dan ibu penulis yang telah mendidik penulis dan senantiasa memberikan dukungan, saran, dan doa,
9. Adik serta seluruh keluarga penulis yang telah membantu dalam bentuk doa, kasih sayang, dan semangat yang tiada hentinya hingga akhirnya skripsi ini dapat terselesaikan,
10. Teman-teman spesial Ahmad Nabih Baril Hilmy dan I Gusti Ngurah Ryo Adi Tarta atas diskusi dan kerja samanya dalam setiap kesempatan,

11. Seluruh teman-teman Lab ICN Fakultas Ilmu Komputer, Universitas Brawijaya yang telah memberikan waktu dan dukungan dalam bentuk semangat serta diskusi selama penulis mengerjakan skripsi,
12. Teman-teman Grup D Beta dan Suwngut Lewle, yang telah menemani perjalanan kuliah penulis sejak semester 1,
13. Teman berproses penulis, Aisyafarras Alfianida S.Pn. yang telah menjadi sebuah tempat hangat untuk bercerita, berkeluh kesah, dan selalu mendorong penulis untuk berusaha lebih baik lagi.

Penulis menyadari bahwa terdapat banyak kekurangan dalam proses penyusunan skripsi ini. Dengan demikian, saran dan kritik yang ditujukan kepada penulis dapat membantu penulis untuk berkembang lebih baik lagi. Penulis berharap penelitian ini dapat memberikan manfaat kepada siapa pun yang menggunakannya.

Malang, 12 Februari 2026



Virgananta Saputra

virganantasinau@gmail.com

ABSTRAK

Virgananta Saputra, Implementasi *Hybrid Intrusion Prevention System (IPS)* Menggunakan Suricata dan Decision Tree Untuk Mencegah *Network Reconnaissance* Berbasis TCP.

Pembimbing: Mahendra Data, S.Kom., M.Kom., Ph.D. dan Dr. Kasyful Amron, S.T., M.Sc.

Peningkatan ancaman siber menuntut mekanisme pertahanan jaringan yang mampu mendeteksi aktivitas *reconnaissance* sejak dini. Sistem pencegahan berbasis *signature* lemah dalam mengenali varian serangan baru atau *zero-day attack*. Penelitian ini mengusulkan solusi melalui pengembangan *Hybrid Intrusion Prevention System* menggunakan Suricata dan Decision Tree. Suricata bertugas sebagai penyaring lapisan pertama sebelum paket dianalisis lebih lanjut oleh model. Metode Decision Tree diterapkan untuk mengklasifikasikan varian serangan yang lolos dari deteksi berbasis *signature*. Lingkungan pengujian dirancang menggunakan mesin virtual untuk mensimulasikan *traffic* normal dan *reconnaissance*. Pengujian kinerja dilakukan dengan membandingkan skenario Suricata tunggal, Decision Tree tunggal, dan sistem hybrid. Hasil eksperimen menunjukkan arsitektur *hybrid* mampu mencapai akurasi sebesar 0.8989 dan recall 0.7249. Sistem ini berhasil memblokir beberapa aktivitas *reconnaissance* yang tidak terdeteksi oleh *ruleset* Suricata. Analisis menunjukkan bahwa Suricata tunggal memiliki *throughput* tinggi tetapi tingkat deteksi serangan rendah. Sebaliknya, sistem *hybrid* mengalami penurunan *throughput* signifikan hingga tersisa rata-rata 9.497 Mbps. Penurunan kinerja jaringan tersebut disebabkan oleh beban komputasi tambahan pada proses klasifikasi Decision Tree. Penelitian ini menyimpulkan bahwa pendekatan *hybrid* meningkatkan keamanan namun sangat memengaruhi kecepatan transmisi.

Kata Kunci: *Network Reconnaissance, Intrusion Prevention System, Hybrid Intrusion Prevention System, Suricata, Decision Tree, TCP*

ABSTRACT

Virgananta Saputra, *Implementation of Hybrid Intrusion Prevention System (IPS) Using Suricata and Decision Tree to Prevent TCP-Based Network Reconnaissance.*

Supervisors: Mahendra Data, S.Kom., M.Kom., Ph.D. dan Dr. Kasyful Amron, S.T., M.Sc.

Escalating cyber threats require a network defense mechanism capable of detecting reconnaissance activities early. Signature-based prevention systems often fail to recognize new attack variants or zero-day attacks. This study proposes a solution by developing a Hybrid Intrusion Prevention System utilizing Suricata and Decision Tree. Suricata functions as the primary filtration layer before packets undergo further analysis by the model. The Decision Tree method is employed to classify attack variants that bypass signature-based detection. A virtual machine environment was designed to simulate both normal traffic and reconnaissance attacks. Performance evaluation involved comparing standalone Suricata, standalone Decision Tree, and hybrid system scenarios. Experimental results demonstrate that the hybrid architecture achieved an accuracy of 0.8989 and a recall of 0.7249. The system successfully blocked some reconnaissance activities undetected by standard Suricata rules. Analysis reveals that while standalone Suricata maintains high throughput, it exhibits low attack detection rates. In contrast, the hybrid system experienced a significant throughput reduction, dropping to an average of 9.497 Mbps. This degradation in network performance is attributed to the additional computational load from the Machine Learning classification process. The study concludes that the hybrid approach enhances security but impacts transmission throughput.

Keywords: Network Reconnaissance, Intrusion Prevention System, Hybrid Intrusion Prevention System, Suricata, Decision Tree, TCP

DAFTAR ISI

PENGESAHAN	ii
PERNYATAAN ORISINALITAS	iii
PRAKATA	iv
ABSTRAK	vi
ABSTRACT	vii
DAFTAR ISI	viii
DAFTAR TABEL	x
DAFTAR GAMBAR	xi
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	2
1.4 Manfaat Penelitian	3
1.5 Batasan Masalah	3
1.6 Sistematika Pembahasan.....	3
BAB 2 LANDASAN KEPUSTAKAAN.....	5
2.1 Kajian Pustaka.....	5
2.2 Dasar Teori.....	8
2.2.1 <i>Reconnaissance</i>	8
2.2.2 <i>Virtual Machine</i>	8
2.2.3 <i>Tcpdump</i>	9
2.2.4 <i>Wireshark</i>	9
2.2.5 <i>Suricata</i>	10
2.2.6 <i>Nfqueue</i>	11
2.2.7 <i>Decision Tree</i>	11
2.2.8 <i>Dataset</i>	12
2.2.9 <i>Confusion Matrix</i>	13
BAB 3 METODOLOGI.....	14
3.1 Studi Literatur	14
3.2 Perancangan Sistem	15
3.2.1 Perancangan Lingkungan Penelitian.....	15

3.2.2	Perancangan Pengumpulan Data	17
3.2.3	Perancangan Ruleset Suricata	20
3.2.4	Perancangan Model Decision Tree	21
3.3	Skenario Pengujian dan Evaluasi	23
3.4	Pengambilan Kesimpulan	26
BAB 4 IMPLEMENTASI		27
4.1	Implementasi Lingkungan Penelitian	27
4.2	Implementasi Pengumpulan Data	29
4.2.1	<i>Traffic Reconnaissance</i>	29
4.2.2	Traffic Normal	37
4.3	Implementasi Ruleset Suricata	42
4.4	Implementasi Model Decision Tree	46
4.4.1	Pemrosesan Data	46
4.4.2	<i>Hyperparameter Tuning</i>	48
4.4.3	Pelatihan dan <i>Deployment</i> Model	50
4.5	Implementasi Skenario Pengujian	55
BAB 5 HASIL DAN ANALISIS		60
5.1	Hasil	60
5.1.1	Hasil Implementasi <i>hybrid</i> IPS	60
5.1.2	Hasil Kinerja IPS	61
5.1.3	Hasil <i>Throughput</i> Jaringan	63
5.2	Analisis	63
5.2.1	Analisis Perbandingan Kinerja IPS	63
5.2.2	Analisis <i>Throughput</i> Jaringan	67
BAB 6 PENUTUP		69
6.1	Kesimpulan	69
6.2	Saran	69
DAFTAR PUSTAKA		71

DAFTAR TABEL

Tabel 2.1 Hasil uji penelitian terdahulu.....	5
Tabel 2.2 Isi <i>confusion matrix</i>	13
Tabel 3.1 Teknik pemindaian pada tahap pengumpulan data	18
Tabel 3.2 Teknik pemindaian pada <i>dataset</i> latih.....	19
Tabel 3.3 Jenis data pada <i>dataset</i> MAWILab	19
Tabel 3.4 Kata kunci pada <i>ruleset</i> Suricata	21
Tabel 3.5 Teknik pemindaian di luar data latih.....	23
Tabel 3.6 Keterangan nilai <i>confusion matrix</i>	25
Tabel 4.1 <i>Script hybrid</i> IPS	28
Tabel 4.2 <i>Pseudocode</i> UDP server	29
Tabel 4.3 <i>Script</i> pemindaian TCP	30
Tabel 4.4 <i>Pseudocode</i> ekstraksi fitur <i>traffic reconnaissance</i>	35
Tabel 4.5 <i>Pseudocode extract</i> filter	37
Tabel 4.6 Potongan <i>syntax</i> filtrasi Tshark	38
Tabel 4.7 <i>Script split</i> pcap.....	39
Tabel 4.8 <i>Script</i> filter pcap.....	39
Tabel 4.9 <i>Script</i> untuk filter protokol TCP dan <i>merge</i>	39
Tabel 4.10 <i>Pseudocode</i> ekstraksi fitur <i>traffic normal</i>	40
Tabel 4.11 Fitur TCP pada data latih <i>reconnaissance</i>	42
Tabel 4.12 <i>Ruleset</i> Suricata untuk teknik pemindaian pada data latih	43
Tabel 4.13 <i>Config</i> Suricata yang digunakan.....	44
Tabel 4.14 <i>Pseudocode preprocessing</i> bagian 1	46
Tabel 4.15 <i>Pseudocode preprocessing</i> bagian 2	47
Tabel 4.16 <i>Pseudocode preprocessing</i> bagian 3	47
Tabel 4.17 Fitur dalam <i>dataset</i> latih model	48
Tabel 4.18 <i>Pseudocode hyperparameter</i> tuning	49
Tabel 4.19 Hasil <i>hyperparameter tuning</i>	50
Tabel 4.20 <i>Pseudocode</i> pelatihan model Decision Tree	50
Tabel 4.21 <i>Pseudocode</i> integrasi dan <i>deployment</i> model.....	52
Tabel 4.22 <i>Script</i> skenario Suricata tunggal.....	56
Tabel 4.23 Konfigurasi Suricata tunggal	56
Tabel 4.24 <i>Script</i> skenario Decision Tree tunggal	57
Tabel 4.25 <i>Script</i> iperf3 pada <i>user</i>	58
Tabel 4.26 <i>Script</i> iperf3 pada <i>target</i>	59
Tabel 5.1 <i>Confusion matrix</i> skenario Suricata tunggal	61
Tabel 5.2 <i>Confusion matrix</i> skenario Decision Tree tunggal.....	62
Tabel 5.3 <i>Confusion matrix</i> skenario <i>hybrid</i> IPS	62
Tabel 5.4 Hasil metrik evaluasi	62
Tabel 5.5 Hasil pengujian <i>throughput</i> tiap skenario	63
Tabel 5.6 Karakteristik teknik pemindaian tidak berhasil dideteksi oleh Suricata	64
Tabel 5.7 Teknik pemindaian yang lolos dari model Decision Tree.....	65

DAFTAR GAMBAR

Gambar 2.1 Antarmuka grafis Wireshark.....	10
Gambar 3.1 Tahapan penelitian	14
Gambar 3.2 Topologi lingkungan penelitian	15
Gambar 3.3 Topologi dengan komponen di dalam mesin IPS	16
Gambar 3.4 Alur paket pada mesin IPS.....	17
Gambar 3.5 Alur paket pada skenario Suricata tunggal	24
Gambar 3.6 Alur paket pada skenario Decision Tree tunggal	24
Gambar 3.7 Alur paket pada Skenario <i>hybrid</i> IPS	25
Gambar 4.1 Versi Kali Linux pada mesin <i>user</i>	27
Gambar 4.2 Versi aplikasi <i>reconnnaissance</i>	27
Gambar 4.3 Konfigurasi IP dan <i>routing</i> pada <i>user</i>	28
Gambar 4.4 Layanan yang berjalan pada target	29
Gambar 4.5 Analisis <i>traffic</i> pemindaian menggunakan Wireshark	42
Gambar 5.1 Aturan iptables pada mesin IPS.....	60
Gambar 5.2 Komponen <i>hybrid</i> IPS yang berjalan	60
Gambar 5.3 Hasil <i>logging</i> dari komponen Suricata.....	61
Gambar 5.4 Hasil <i>logging</i> komponen Decision Tree	61
Gambar 5.5 Perbandingan metrik evaluasi pada setiap skenario.....	64
Gambar 5.6 Perbandingan paket <i>reconnnaissance</i> dengan paket normal	66
Gambar 5.7 Perbandingan rata-rata <i>throughput</i> pada setiap skenario	67

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan pesat layanan digital di Indonesia berbanding lurus dengan peningkatan risiko keamanan siber. Peningkatan risiko ini terlihat dari laporan tahunan yang di rilis oleh Badan Siber dan Sandi Negara (BSSN). Menurut BSSN (2024), tercatat sebanyak 330527636 *traffic* anomali sepanjang tahun 2024. Mirai Botnet menjadi ancaman paling banyak di antara *traffic* anomali tersebut. Skala ancaman ini menegaskan urgensi untuk membangun arsitektur pertahanan siber yang sistematis dan proaktif. Namun, diperlukan pemahaman terhadap langkah-langkah serangan siber terlebih dahulu sebelum membangun arsitektur pertahanan.

Reconnaissance merupakan tahap awal yang krusial sebelum penyerang melakukan eksploitasi (Andress dan Winterfeld, 2014) dalam (Hutchins, Cloppert dan Amin, 2011). Pada model Cyber Kill Chain, *reconnaissance* dijelaskan sebagai sebuah langkah pencarian informasi spesifik pada suatu target (Hutchins, Cloppert dan Amin, 2011). Model Cyber Kill Chain menjelaskan bahwa kegagalan di satu tahap serangan dapat menggagalkan keseluruhan serangan (Hutchins, Cloppert dan Amin, 2011). Penelitian terdahulu oleh Roy dkk. (2022) juga menegaskan bahwa informasi pada tahap *reconnaissance* dapat meningkatkan tingkat kesuksesan serangan. *Network reconnaissance* merupakan salah satu kategori *reconnaissance*. *Network reconnaissance* dapat memberikan informasi mengenai infrastruktur target dan juga kerentanan yang dimiliki (Achleitner dkk., 2017). Penjelasan dari Roy dkk. (2022) dan Hutchins dkk. (2011) menandakan jika aktivitas *reconnaissance* dapat dideteksi dan dihentikan sedini mungkin, peluang terjadinya eksploitasi lebih lanjut dapat dicegah secara signifikan. Untuk seterusnya, kata *reconnaissance* pada artikel ini akan merujuk pada *network reconnaissance*.

Salah satu pendekatan pertahanan siber yang banyak digunakan adalah sistem deteksi berbasis *signature* seperti Suricata (Asad dan Gashi, 2022). Menurut Kaur dan Singh (2013), *signature* adalah pola pada suatu serangan yang sudah diketahui. Suricata dapat berjalan sebagai 2 jenis sistem. Sistem deteksi atau *Intrusion Detection System* (IDS) dan sistem pencegahan atau *Intrusion Prevention System* (IPS). IPS adalah sistem keamanan jaringan yang mendeteksi aktivitas berbahaya dan secara otomatis mencegahnya sesuai kebijakan yang ditetapkan (Erney dan Chowdhury, 2022). Suricata dapat mengidentifikasi *traffic* berbahaya melalui *ruleset* yang telah didefinisikan. *Ruleset* mengacu pada kumpulan aturan berbasis *signature* yang memuat pola *traffic* dan indikator serangan yang dapat dianalisis oleh Suricata (Asad dkk., 2024). Namun, metode ini memiliki ketergantungan tinggi pada pola serangan yang sudah dikenal. Suricata rentan terhadap serangan varian baru atau dikenal juga *zero-day attack* (Asad dkk., 2024). Menurut Bilge dan Dumitraş (2012), *zero-day attack* merupakan serangan yang memanfaatkan kerentanan yang belum diketahui oleh publik. Keterbatasan

Suricata mendeteksi serangan *zero-day attack* membuat adanya jeda waktu dari saat serangan tersebut terjadi dan *ruleset* dibuat (Asad dkk., 2024). Temuan oleh Asad dkk. (2024) menegaskan kebutuhan arsitektur yang tidak hanya mengandalkan *ruleset*.

Machine Learning memiliki kemampuan untuk memodelkan pola anomali dan variasi serangan baru yang tidak bisa diidentifikasi oleh *ruleset*. *Machine Learning* (ML) adalah cabang kecerdasan buatan yang membangun model komputasi untuk belajar dari data dan membuat prediksi secara mandiri (Kufel dkk., 2023). Sebuah penelitian oleh Guo (2023) menyebutkan bahwa pendekatan ML cukup menjanjikan untuk mendeteksi serangan *zero-day*. Namun, pengembangan ML perlu dilakukan secara berhati-hati guna mengelola *false positive* dan biaya komputasi. Salah satu contoh model ML adalah Decision Tree. Model Decision Tree mendapatkan kinerja paling baik di antara model lain dalam kasus mendeteksi *reconnaissance* (Bertoli dkk., 2021).

Kesenjangan penelitian timbul dari keterbatasan individual kedua pendekatan. IPS berbasis *ruleset* seperti Suricata efektif untuk serangan yang telah diketahui namun gagal mendeteksi ancaman *zero-day*. Disisi lain, ML mampu mengenali anomali baru tetapi berpotensi menghasilkan tingkat *false positive* yang tinggi. Tingkat *false positive* yang tinggi tersebut menjadi masalah krusial jika diterapkan langsung pada sistem IPS. Masalah ini dikhawatirkan dapat menyebabkan pemblokiran *traffic* yang sah. Oleh karena itu, penelitian ini mengusulkan penggabungan kedua pendekatan sebagai urgensi untuk menciptakan arsitektur pertahanan *hybrid*. Pendekatan *hybrid* yang diusulkan didukung oleh penelitian Syamsuddin dan Barukab (2022) yang berhasil menggabungkan Suricata dan ML sebagai IDS. Penelitian yang akan dilakukan mengadaptasi model *hybrid* tersebut namun mengembangkannya secara khusus menjadi IPS. Pengembangan *Hybrid Intrusion Prevention System* (IPS) bertujuan memanfaatkan kecepatan dari Suricata sekaligus menutupi kelemahannya menggunakan ML untuk mencegah aktivitas *reconnaissance*.

1.2 Rumusan Masalah

Rumusan masalah pada penelitian ini adalah sebagai berikut:

1. Bagaimana mengimplementasikan arsitektur *hybrid Intrusion Prevention System* yang mengombinasikan Suricata dengan Decision Tree?
2. Bagaimana perbandingan kinerja antara arsitektur *hybrid Intrusion Prevention System* dengan arsitektur Suricata tunggal atau Decision Tree tunggal?

1.3 Tujuan Penelitian

Tujuan dari penelitian ini adalah merancang, mengimplementasikan, dan mengevaluasi arsitektur *hybrid IPS* yang mengombinasikan Suricata dengan model Decision Tree untuk mencegah aktivitas *reconnaissance*. Optimalisasi yang dituju

meliputi peningkatan efektivitas pencegahan, reliabilitas keputusan, serta efisiensi operasional jaringan. Penelitian ini juga bertujuan menghasilkan perbandingan yang terukur antara arsitektur *hybrid IPS* dan arsitektur Suricata atau Decision Tree tunggal berdasarkan kinerja deteksi dan *throughput* jaringan. Dengan demikian, penelitian ini diharapkan menghadirkan rancangan arsitektur yang andal beserta bukti empiris mengenai *trade-off* kinerja dengan *throughput* jaringan.

1.4 Manfaat Penelitian

Manfaat dari penelitian ini adalah tersedianya pendekatan pencegahan dini terhadap serangan siber melalui pemutusan tahapan *reconnaissance*. Dengan begitu, risiko eskalasi ke eksploitasi yang lebih merusak dapat dihindari. Arsitektur *hybrid IPS* yang diusulkan diharapkan meningkatkan kualitas layanan jaringan melalui deteksi yang lebih andal. Penelitian ini menghasilkan artefak implementasi berupa *ruleset* Suricata dan model Decision Tree terlatih yang dapat diadopsi dan di replikasi sebagai rujukan teknis. Hasil pengukuran komprehensif yang disajikan dapat menjadi dasar pengambilan keputusan untuk perencanaan kapasitas, kebijakan keamanan jaringan, serta pengembangan riset lanjutan pada topik *hybrid intrusion detection/prevention*.

1.5 Batasan Masalah

Batasan masalah yang diterapkan pada penelitian ini adalah sebagai berikut:

1. Simulasi serangan *reconnaissance* menggunakan aplikasi Nmap, Unicornscan, Hping3, dan Masscan dengan protokol TCP.
2. Perbandingan kinerja yang dihitung menggunakan *confusion matrix* dan metrik evaluasi seperti akurasi, presisi, *recall*, *f1-score* dan *throughput* jaringan.
3. Implementasi lingkungan penelitian menggunakan Wireguard untuk memungkinkan pengaturan *traffic*. Namun, tidak dijelaskan lebih lanjut pada penelitian ini.

1.6 Sistematika Pembahasan

Sistematika pembahasan pada skripsi ini adalah sebagai berikut.

BAB 1 PENDAHULUAN

Bab pertama ini membangun fondasi penelitian dengan menguraikan latar belakang masalah, rumusan masalah, serta tujuan yang ingin dicapai. Selain itu, bab ini juga menetapkan batasan masalah, menjelaskan manfaat yang diharapkan dari penelitian, dan menyajikan sistematika pembahasan secara keseluruhan.

BAB 2 LANDASAN KEPUSTAKAAN

Bab ini mengulas berbagai landasan teori dan studi terdahulu yang relevan. Pembahasan dimulai dengan identifikasi celah penelitian yang menjadi justifikasi

bagi metode yang diusulkan. Kemudian kajian ini ditutup dengan dari konsep penting *reconnaissance*, komponen lingkungan penelitian, pendekatan deteksi berbasis *signature* menggunakan Suricata, model ML, dan mekanisme Nfqueue.

BAB 3 METODOLOGI

Bab ketiga merinci langkah-langkah metodologis yang ditempuh secara sistematis. Penjelasan di dalamnya mencakup penentuan tipe penelitian, perancangan skenario eksperimen dan lingkungan uji, prosedur pengumpulan data, tahapan pembentukan *dataset*, proses pelatihan model, serta strategi integrasi sistem *hybrid*. Bab ini juga menjelaskan rencana pengujian beserta metrik evaluasi yang akan digunakan.

BAB 4 IMPLEMENTASI

Bab keempat mendokumentasikan proses realisasi teknis dari seluruh rancangan yang telah diuraikan pada bab sebelumnya. Bagian ini merinci konfigurasi perangkat keras dan lunak dalam mengimplementasikan sistem IPS *hybrid*. Setelah rancangan berhasil diimplementasikan, Kinerja sistem akan diuji.

BAB 5 HASIL DAN ANALISIS

Pada bab ini akan disajikan hasil dari serangkaian pengujian yang telah dilakukan, diikuti dengan analisis mendalam. Penyajian data mencakup hasil evaluasi kinerja deteksi dan *throughput* jaringan.

BAB 6 PENUTUP

Bab terakhir ini menyajikan rangkuman dari keseluruhan temuan penelitian dalam bentuk kesimpulan. Selain itu, disampaikan pula beberapa saran yang dapat menjadi acuan untuk pengembangan penelitian berikutnya.

BAB 2 LANDASAN KEPUSTAKAAN

Landasan kepastakaan berisi uraian dan pembahasan tentang teori, konsep, model, metode, atau sistem dari pustaka ilmiah. Uraian dan pembahasan tersebut berkaitan dengan tema, masalah, atau pertanyaan penelitian. Pada landasan kepastakaan terdapat landasan teori dari berbagai sumber pustaka yang terkait dengan teori dan metode yang digunakan dalam penelitian.

2.1 Kajian Pustaka

Terdapat beberapa penelitian terdahulu terkait dengan arsitektur *hybrid*. Berikut beberapa penelitian yang disajikan dalam paragraf dan Tabel 2.1 mengenai masalah, metode, hasil, dan relevansi penelitian.

Tabel 2.1 Hasil uji penelitian terdahulu

Nama dan Tahun	Masalah	Metode	Hasil	Relevansi
Syamsuddin dan Barukab (2022)	IDS konvensional sering gagal mendeteksi varian serangan <i>botnet</i> baru pada jaringan IoT. Algoritma kNN standar memiliki kinerja lambat saat memproses <i>dataset</i> serangan <i>botnet</i> yang besar.	Menggabungkan teknik seleksi fitur pada kNN dan integrasi Suricata. Model kNN dan Suricata IDS diimplementasikan ke dalam perangkat Raspberry Pi.	Integrasi kNN dengan <i>Forward Selection</i> dan Suricata menghasilkan akurasi deteksi sebesar 99,89 persen.	Penelitian ini menjadi acuan integrasi Suricata dan ML. Namun, penelitian yang akan dilakukan mengembangkan sistem menjadi IPS.
Coutinho dan Araújo, (2025)	Sistem keamanan konvensional sering gagal mendeteksi serangan yang terus berkembang serta menghasilkan	Penelitian ini menerapkan algoritma <i>Machine Learning</i> sebagai salah satu komponen mendeteksi serangan.	Algoritma <i>Machine Learning</i> berhasil mendeteksi serangan dengan kinerja yang sangat baik. Hasil deteksi secara	Penelitian yang akan dilakukan menggunakan arsitektur komponen yang lebih sederhana dengan algoritma

	banyak <i>false positive</i> . Kurangnya integrasi antara sistem deteksi dan mekanisme respons menghambat penanggulangan ancaman siber.	Indikator ancaman yang tervalidasi dikonversi otomatis menjadi <i>ruleset</i> pemblokiran berbasis IP pada Suricata.	otomatis diubah menjadi <i>ruleset</i> IPS untuk memblokir IP sumber serangan.	Decision Tree sebagai pengambil keputusan final setelah penyaringan oleh Suricata.
--	---	--	--	--

Penelitian Syamsuddin dan Barukab (2022) menyoroti permasalahan pada ekosistem *Internet of Things* (IoT) yang rentan terhadap serangan *malware botnet*. Pertumbuhan perangkat IoT yang masif sering kali tidak terlindungi oleh sistem keamanan memadai. Sistem IDS tradisional berbasis *signature* dinilai kurang efektif dalam mengenali varian serangan baru yang terus berevolusi. Peneliti memilih algoritma k-Nearest Neighbor (kNN) karena karakteristiknya yang sederhana dan tangguh. Namun, kNN memiliki kelemahan utama yakni kinerja yang lambat saat memproses *dataset* berukuran besar. Beban komputasi tinggi kNN menjadi kendala serius ketika diterapkan pada perangkat dengan sumber daya terbatas seperti Raspberry Pi.

Solusi yang ditawarkan oleh Syamsuddin dan Barukab (2022) berfokus pada peningkatan efisiensi kNN melalui penerapan teknik seleksi fitur. Penelitian tersebut membandingkan tiga teknik seleksi fitur bawaan pada Rapidminer, yaitu *Information Gain*, *Forward Selection*, dan *Backward Elimination*. Eksperimen dilakukan menggunakan *dataset* Bot-IoT yang merepresentasikan lalu lintas jaringan IoT. Tujuan utama eksperimen adalah menemukan kombinasi fitur yang mampu mempertahankan akurasi tinggi sekaligus meminimalkan waktu pemrosesan. Model terbaik dari hasil komparasi kemudian diintegrasikan dengan mesin Suricata IDS untuk membentuk sistem bernama SUKRY (Suricata IDS with Enhanced kNN Algorithm on Raspberry Pi).

Hasil penelitian menunjukkan bahwa kNN dengan teknik *Forward Selection* (kNN-FS) memberikan hasil paling optimal dibandingkan teknik lainnya. Model kNN-FS menghasilkan tingkat akurasi tertinggi sebesar 99,89% dalam mendeteksi serangan *botnet*. Selain itu, kNN-FS juga mendapatkan waktu eksekusi tercepat yaitu 7 detik. Kinerja waktu eksekusi kNN-FS jauh lebih efisien dibandingkan kNN dengan *Backward Elimination* yang membutuhkan waktu 18 detik. Syamsuddin dan Barukab (2022) menyimpulkan bahwa SUKRY mampu beroperasi secara efektif sebagai sistem IDS cerdas pada Raspberry Pi.

Penelitian yang dilakukan oleh Syamsuddin dan Barukab (2022) memiliki relevansi erat dengan penelitian yang akan dilakukan. Penelitian yang akan

dilakukan mengadopsi pendekatan integrasi antara Suricata dengan algoritma ML untuk meningkatkan kinerja deteksi ancaman. Meskipun demikian, terdapat perbedaan mendasar pada mekanisme penanganan ancaman antara kedua penelitian. Syamsuddin dan Barukab (2022) membatasi ruang lingkup sistem sebagai IDS yang berfungsi memberi peringatan. Sebaliknya, penelitian yang akan dilakukan mengembangkan konsep tersebut menjadi IPS. Pengembangan ini memungkinkan deteksi sekaligus pemblokiran serangan secara otomatis.

Coutinho dan Araújo (2025) mengangkat permasalahan mengenai ancaman siber yang semakin canggih. Sistem keamanan tradisional sering kali memiliki keterbatasan kemampuan adaptasi terhadap pola serangan baru. Sistem keamanan tradisional juga kerap menghasilkan tingkat *false positive* yang tinggi sehingga menyulitkan tim keamanan. Selain itu, kurangnya integrasi antar-alat keamanan menyebabkan operasi pertahanan serangan berjalan secara terisolasi dan menghambat respons insiden yang efektif.

Penelitian Coutinho dan Araújo (2025) mengusulkan Modular Intelligent Cybersecurity Response Architecture (MICRA) sebagai kerangka pertahanan siber yang adaptif. MICRA mengintegrasikan enam modul terpisah yang mencakup pengumpulan data, analisis ancaman, dan otomatisasi respons. Metode MICRA menggabungkan teknik ML, analisis heuristik, dan analisis perilaku *traffic* jaringan untuk mengidentifikasi anomali pada *traffic* jaringan. Arsitektur MICRA menerapkan mekanisme validasi berlapis sebelum mengeksekusi tindakan mitigasi untuk memastikan akurasi keputusan. Sistem secara otomatis memperbarui aturan keamanan jaringan untuk memblokir akses dari sumber IP berbahaya. Aturan blokir secara otomatis diterapkan pada *firewall* dan komponen IPS.

Implementasi MICRA terbukti efektif dalam memperkuat deteksi serta merespons ancaman siber. Pengujian berbagai algoritma ML pada *dataset* CIC-IDS 2017 menghasilkan kinerja klasifikasi yang sangat baik. Sistem MICRA berhasil mengubah indikator serangan menjadi aturan blokir pada *firewall* dan IPS untuk memblokir IP penyerang secara otomatis. Mekanisme validasi berlapis pada MICRA terbukti mampu meminimalkan risiko kesalahan pemblokiran pada *traffic* yang sah.

Penelitian Coutinho dan Araújo (2025) memiliki relevansi yang signifikan dengan penelitian yang akan dilakukan. Relevansi ini khususnya dalam penerapan konsep validasi berlapis. Mekanisme pemblokiran berbasis *signature* dan ML akan diimplementasikan namun dengan arsitektur komponen yang lebih sederhana. Fokus penelitian terletak pada penggabungan langsung antara Suricata sebagai penyaring berbasis *ruleset* dan ML sebagai lapisan analisis lanjutan. Model ML yang menggunakan Decision Tree akan berfungsi sebagai pengambil keputusan final terhadap paket yang telah melewati *ruleset* Suricata. Implementasi mekanisme pemblokiran akan dilakukan dengan memblokir paket, dibandingkan MICRA yang melakukan pemblokiran IP.

2.2 Dasar Teori

Sub bab ini membahas mengenai dasar teori yang digunakan sebagai landasan teori penelitian yang akan digunakan. Penjelasan mengenai *reconnaissance* berperan untuk memperkuat landasan mengenai ancaman yang akan dicegah. Penjelasan mengenai *virtual machine*, *Tcpdump* dan *Wireshark* menjelaskan lingkungan dan alat yang digunakan selama penelitian. Penjelasan mengenai *Suricata*, *Nfqueue*, *Decision Tree*, *dataset*, dan *confusion matrix* menjadi penjelasan mengenai komponen sistem *hybrid IPS* yang diusulkan dan teknik evaluasinya.

2.2.1 Reconnaissance

Reconnaissance merupakan tahap awal yang krusial dalam siklus serangan siber. Pada tahap ini penyerang berfokus mengumpulkan informasi mengenai target sebelum melakukan eksploitasi lebih lanjut. Dalam model Cyber Kill Chain, *reconnaissance* dijelaskan sebagai aktivitas pencarian informasi spesifik terkait sistem, jaringan, atau aplikasi yang menjadi sasaran serangan (Hutchins, Cloppert dan Amin, 2011). Model tersebut juga menegaskan bahwa kegagalan pada salah satu tahap termasuk *reconnaissance* dapat menggagalkan keseluruhan rangkaian serangan (Hutchins, Cloppert dan Amin, 2011). Kegagalan serangan dapat terjadi dikarenakan setiap tahap saling bergantung satu sama lain.

Aktivitas *reconnaissance* dapat dilakukan secara pasif melalui pengumpulan informasi *open source*, maupun secara aktif (Roy dkk., 2022). Metode aktif dapat dilakukan dengan melakukan pemindaian jaringan untuk mengidentifikasi IP, *port* terbuka, dan layanan yang tersedia pada target (Roy dkk., 2022). Roy dkk. (2022) menekankan bahwa setiap informasi yang dikumpulkan pada tahap *reconnaissance* akan meningkatkan probabilitas keberhasilan serangan lanjutan. Contoh serangan lanjutan dapat berupa eksploitasi kerentanan dan pengambilalihan sistem. Analisis mendalam membutuhkan sebuah lingkungan pengujian terisolasi yang diimplementasikan melalui *virtual machine*.

2.2.2 Virtual Machine

Virtual machine merupakan sistem komputer logis yang dibentuk oleh lapisan aplikasi. Sistem ini memungkinkan beberapa sistem operasi dapat berjalan di atas satu mesin fisik. Menurut Popek dan Goldberg (1974), *virtual machine monitor* menciptakan lingkungan eksekusi untuk *virtual machine*. Lingkungan eksekusi yang tercipta setara dengan mesin asli namun terkontrol oleh aplikasi pengelola sumber daya. *Virtual machine* tidak berinteraksi langsung dengan perangkat keras, melainkan dengan aplikasi yang meniru fungsi perangkat keras tersebut (Langer dan French, 2011). Huawei Technologies Co., Ltd. (2023) mendefinisikan *virtual machine* sebagai sistem komputer logis yang berjalan dalam lingkungan terisolasi dan memiliki fungsi perangkat keras lengkap. Dalam arsitektur ini dikenal istilah *host* sebagai

mesin fisik, *guest* sebagai *virtual machine*, serta *hypervisor* atau *virtual machine monitor* sebagai lapisan pengelola di antaranya.

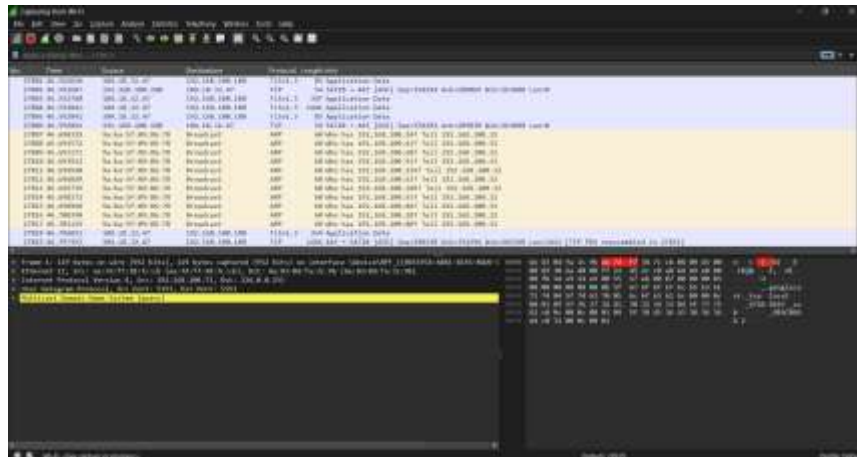
Karakteristik utama *virtual machine* terletak pada kemampuan menjalankan beberapa *guest* terisolasi dalam satu *host*. Medina dan García (2014) menegaskan bahwa isolasi bertujuan mencegah dominasi konsumsi sumber daya oleh sebuah *virtual machine*. Mekanisme isolasi ini menjamin bahwa proses pada suatu *guest* tidak akan mengganggu kinerja *guest* lainnya. Fleksibilitas tersebut sangat krusial dalam skenario pengelolaan *traffic* data yang kompleks dan dinamis. *Guest* dapat difungsikan mengatur *routing* guna mengarahkan *traffic* sesuai kebutuhan. Kapabilitas ini memungkinkan pengambilan keputusan terhadap penerusan paket berdasarkan kebijakan jaringan yang telah ditentukan. Selain itu, setiap *guest* dapat dikonfigurasi secara independen untuk menjalankan layanan spesifik sesuai kebutuhan.

2.2.3 Tcpdump

Tcpdump merupakan salah satu aplikasi berbasis baris perintah yang digunakan untuk menyimpan dan menampilkan *traffic* jaringan. Menurut Goyal dan Goyal (2017), Tcpdump termasuk aplikasi yang ringan dan efektif untuk mempelajari protokol TCP/IP. Tcpdump menyimpan paket jaringan dan dapat dikonfigurasi dengan aturan penyaringan tertentu. Dengan begitu, penyimpanan *traffic* dapat difokuskan pada *traffic* relevan sesuai kebutuhan penelitian (Goyal dan Goyal, 2017). Tcpdump dapat berfungsi sebagai *packet sniffer* yang menyimpan paket dalam format berkas pcap. Penyimpanan dalam format pcap memungkinkan *traffic* dianalisis lebih lanjut menggunakan aplikasi lain maupun tahapan pemrosesan berikutnya.

2.2.4 Wireshark

Wireshark merupakan aplikasi *packet analyzer* yang digunakan untuk menyimpan, menampilkan, dan menganalisis data *traffic*. Menurut Goyal dan Goyal (2017), Wireshark termasuk salah satu aplikasi *packet sniffing* paling populer. Wireshark menyediakan antarmuka grafis yang memudahkan pemantauan *traffic* jaringan dan analisis protokol. Contoh antarmuka grafis pada Wireshark ditunjukkan pada Gambar 2.1. Wireshark mampu membaca berkas pcap, kemudian melakukan *decode* setiap lapisan protokol. Dengan begitu, informasi sebuah paket seperti *header* dan *payload* paket dapat diamati. Wireshark juga dapat menerapkan berbagai filter untuk menyaring data *traffic* yang tidak relevan.



Gambar 2.1 Antarmuka Wireshark

Selain menyediakan antarmuka grafis, Wireshark juga dilengkapi dengan alat analisis berbasis baris perintah yang dikenal sebagai TShark. TShark memungkinkan pengolahan dan analisis *traffic* tanpa ketergantungan pada tampilan visual. TShark mampu menjalankan berbagai tugas manipulasi data *traffic* secara otomatis melalui sebuah *script*. Fungsinya mencakup pemecahan berkas pcap menjadi segmen lebih kecil, filtrasi *traffic*, dll. Kapabilitas ini menjadikan proses analisis *traffic* lebih fleksibel dan dapat dijalankan pada lingkungan yang tidak memiliki antarmuka grafis. Hasil analisis *traffic* dapat menjadi landasan krusial dalam pengembangan arsitektur pertahanan jaringan.

2.2.5 Suricata

Suricata merupakan salah satu sistem deteksi intrusi jaringan berbasis *signature* yang banyak digunakan (Asad, Adhikari dan Gashi, 2024). Suricata bekerja dengan cara menerapkan *ruleset* berisi *signature* untuk melakukan deteksi serangan (Asad dan Gashi, 2022). *Ruleset* tersebut memanfaatkan kombinasi seperti protokol, *port*, IP, serta pola *payload*. Keseluruhan kombinasi memungkinkan Suricata mendeteksi berbagai serangan yang telah terdokumentasi sebelumnya (Asad, Adhikari dan Gashi, 2024).

Suricata menyediakan dua mekanisme *logging* aktivitas, yaitu fast log dan EVE log. Fast log dirancang untuk menyajikan format peringatan ringkas yang memuat informasi esensial mengenai *signature* yang terdeteksi. Di sisi lain, EVE log menawarkan *logging* yang lebih komprehensif berbasis format JSON (Asad, Adhikari dan Gashi, 2024). *Logging* yang dapat dicatat oleh EVE log mencakup *metadata* detail, statistik *runtime* Suricata, dll. Kedua jenis *logging* digunakan dalam penelitian ini untuk mendokumentasikan paket yang diblokir oleh Suricata. Dokumentasi ini krusial sebagai data untuk memvalidasi efektivitas *ruleset* yang diterapkan.

Suricata dapat dioperasikan dalam mode pasif sebagai IDS yang hanya menghasilkan peringatan. Selain itu, ada mode *inline* sebagai IPS yang mampu

secara langsung menghentikan paket berbahaya sebelum mencapai target (Open Information Security Foundation (OISF), 2025). Pembaruan *signature* pada *ruleset* menjadi faktor krusial bagi Suricata dalam menghadapi perkembangan varian serangan siber (Asad, Adhikari dan Gashi, 2024). Ketergantungan pada *signature* membuat pendekatan ini rentan terhadap serangan *zero-day*.

Penelitian Liao dkk. (2020) menunjukkan bahwa *ruleset* populer seperti ET OPEN masih memiliki kelemahan terhadap aktivitas *reconnaissance*. Kinerja deteksi *ruleset* memperoleh hasil rendah khususnya pada skenario teknik *evasion* (Liao dkk., 2020). Keterbatasan ini mendorong pengembangan *ruleset* baru, tetapi proses tersebut membutuhkan alokasi waktu (Asad, Adhikari dan Gashi, 2024). Oleh karena itu, diperlukan sebuah pendekatan baru untuk membantu Suricata. Pendekatan ini disebut juga arsitektur *hybrid*, yang menggabungkan Suricata dengan model ML. Arsitektur *hybrid* membuat Suricata berperan sebagai lapisan pertama untuk menyaring serangan dengan *signature* yang sudah diketahui. Sedangkan lapisan berikutnya berfokus mendeteksi pola anomali yang belum diketahui (Ouiazzane, Addou dan Barramou, 2022). Implementasi arsitektur *hybrid* memerlukan metode integrasi yang tepat agar kedua lapisan dapat berkolaborasi secara efektif.

2.2.6 Nfqueue

Nfqueue adalah sebuah target pada iptables atau nftables yang berfungsi menyalurkan paket dari *kernel* ke aplikasi di tingkat *userspace*. Aplikasi dapat menganalisis dan memutuskan nasib paket tersebut melalui mekanisme antrean (Netfilter Project, 2025a). Ketika sebuah aturan iptables atau nftables mengarah ke Nfqueue, paket tidak langsung diproses oleh sistem operasi. Paket akan dimasukkan ke dalam antrean tertentu yang dapat diambil oleh aplikasi untuk diproses (Netfilter Project, 2025a). Antarmuka ini disediakan oleh komponen `libnetfilter_queue` pada tingkat *userspace*. Komponen `libnetfilter_queue` memungkinkan aplikasi membaca paket dan mengembalikan keputusan berupa tolak, terima atau *reinject* (Netfilter Project, 2025b).

Dokumentasi Suricata menjelaskan bahwa Nfqueue merupakan salah satu mekanisme utama untuk menjalankan Suricata dalam mode *inline*. Pada mode *inline*, paket yang diarahkan ke sebuah antrean Nfqueue akan diambil Suricata untuk dianalisis berdasarkan *ruleset* sebelum diteruskan atau ditolak (Open Information Security Foundation (OISF), 2025). Paket yang tidak ditolak oleh Suricata kemudian dilanjutkan ke antrean Nfqueue berikutnya. Paket akan dianalisis oleh komponen lain pada antrean berikutnya sebagai mekanisme pertahanan berlapis.

2.2.7 Decision Tree

Decision tree merupakan salah satu metode klasifikasi yang merepresentasikan proses pengambilan keputusan ke dalam struktur

menyerupai pohon. Arsitektur model ini tersusun atas tiga komponen utama yaitu simpul akar, simpul internal, dan simpul daun. Menurut Safavian dan Landgrebe (1991), Decision Tree bekerja dengan membagi ruang data secara *recursive* berdasarkan atribut paling informatif. Proses pembagian dilakukan berulang kali sehingga tiap simpul daun merepresentasikan label kelas tertentu. Decision Tree dapat digunakan untuk masalah klasifikasi dengan kinerja yang baik dan *interpretability* yang tinggi dibandingkan metode ML lain (Breiman dkk., 1984). Hasil penelitian Bertoli dkk., (2021) menunjukkan bahwa Decision Tree unggul dalam mendeteksi aktivitas *reconnaissance* dibandingkan model lain.

Proses pemilihan atribut pemisah dalam penelitian ini didasarkan pada perhitungan nilai Gini *Impurity* atau indeks Gini. Metrik statistik ini berfungsi mengukur tingkat kemurnian atau diversitas kelas dalam sebuah simpul data. Nilai Gini nol mengindikasikan kemurnian sempurna di mana seluruh sampel dalam simpul memiliki kelas yang sama (Olivier dan Aldrich, 2021). Sebaliknya, nilai Gini akan meningkat seiring bertambahnya keberagaman label kelas pada suatu himpunan data. Berdasarkan Breiman dkk. (1984), perhitungan indeks Gini untuk sebuah himpunan data dirumuskan melalui Persamaan 2.1.:

$$Gini(D) = 1 - \sum_{i=1}^c (P_i)^2 \quad (2.1)$$

Variabel P_i merepresentasikan proporsi sampel yang termasuk dalam kelas ke- i dari total kelas c yang tersedia. Penjumlahan kuadrat peluang digunakan untuk mengestimasi probabilitas terpilihnya dua sampel dengan kelas yang identik. Pengurangan terhadap angka satu dilakukan guna merepresentasikan tingkat probabilitas terjadinya kesalahan klasifikasi pada simpul tersebut.

Decision Tree melakukan iterasi pada setiap atribut untuk mencari pemisahan dengan nilai Gini *Impurity* terendah. Atribut dengan nilai Gini terkecil dipilih karena dianggap paling mampu memisahkan kelas secara homogen (Breiman dkk., 1984). Penurunan nilai Gini menandakan bahwa data pada simpul anak telah terkelompokkan menjadi lebih teratur. Mekanisme ini memastikan pohon keputusan tumbuh dengan memisahkan kelas normal dan *reconnaissance* secara efektif. Selain itu, perhitungan Gini dinilai lebih efisien secara komputasi dibandingkan entropi karena tidak melibatkan operasi logaritma. Proses pembentukan struktur Decision Tree tersebut sangat bergantung pada kualitas *dataset* yang digunakan selama pelatihan.

2.2.8 Dataset

Dataset adalah sekumpulan data yang saling berhubungan dan dikelola untuk tujuan tertentu (Renear, Sacchi dan Wickett, 2010). *Dataset* dapat diartikan berbeda dalam konteks ML. Menurut Mao dkk. (2023), *dataset* adalah sekumpulan data yang digunakan untuk melatih dan melakukan

evaluasi terhadap model ML. *Dataset* umumnya terdiri atas kombinasi *traffic* normal dan *traffic* serangan untuk pengembangan IDS/IPS. Perbedaan keduanya membuat model dapat belajar membedakan pola *traffic* normal dari pola anomali yang merepresentasikan aktivitas serangan. Pendekatan ini memungkinkan model mempelajari korelasi antara fitur pada *traffic* dengan kategori serangan atau kondisi normal (Bertoli dkk., 2021). Dengan model yang sudah siap dilatih, diperlukan sebuah metode pengujian terhadap model.

2.2.9 Confusion Matrix

Confusion matrix merupakan tabel dua dimensi yang merangkum kinerja model klasifikasi dengan membandingkan kelas aktual dan kelas yang diprediksi. Menurut Ting (2011), setiap baris merepresentasikan kelas aktual, sedangkan setiap kolom merepresentasikan kelas prediksi. Tabel 2.2 memuat isi dari *True Positive (TP)*, *False Positive (FP)*, *True Negative (TN)*, dan *False Negative (FN)*.

Tabel 2.2 Isi *confusion matrix*

Aktual/Prediksi	Prediksi Positif	Prediksi Negatif
Aktual Positif	TP	FP
Aktual Negatif	FN	TN

Confusion matrix menjadi dasar perhitungan berbagai metrik evaluasi seperti akurasi, presisi, *recall*, dan *f1-score* (Sathyanarayanan dan Tantri, 2024). Rumus menghitung metrik evaluasi hasil turunan *confusion matrix* terdapat pada Persamaan 2.2 hingga 2.5. Metrik evaluasi ini digunakan untuk memberikan gambaran terhadap kesalahan pada tiap kelas.

$$Akurasi = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.2)$$

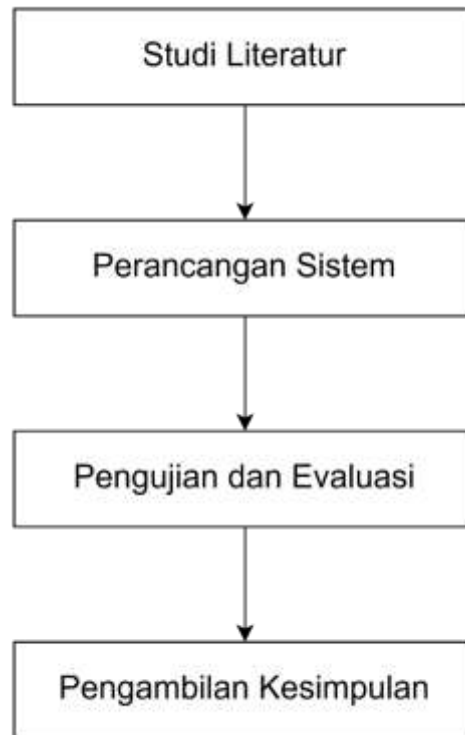
$$Presisi = \frac{TP}{TP + FP} \quad (2.3)$$

$$Recall = \frac{TP}{TP + FN} \quad (2.4)$$

$$F1 - Score = \frac{2 * Presisi * Recall}{FP + TN} \quad (2.5)$$

BAB 3 METODOLOGI

Bab ini menguraikan metodologi penelitian yang diterapkan untuk mencapai tujuan yang telah ditetapkan. Rangkaian penelitian ini mencakup beberapa tahapan sistematis yang dimulai dari studi literatur, perancangan sistem, skenario pengujian dan evaluasi, hingga pengambilan kesimpulan. Alur kerja penelitian secara keseluruhan divisualisasikan pada Gambar 3.1.



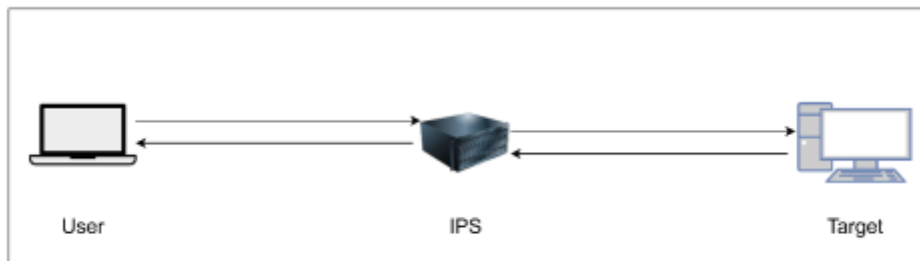
Gambar 3.1 Tahapan penelitian

3.1 Studi Literatur

Tahap studi literatur dilaksanakan guna membangun landasan teoretis mengenai fenomena deteksi anomali pada jaringan. Fokus utama dalam proses ini adalah mendalami fase *reconnaissance* yang menjadi tahap awal dalam siklus serangan. Penggunaan Suricata dikaji sebagai sistem deteksi berbasis *signature* serta mengidentifikasi berbagai keterbatasan di dalamnya. Literatur mengenai pemanfaatan ML turut dipelajari untuk menutupi keterbatasan Suricata dengan metode analisis statistik *traffic*. Penggabungan kedua metode ini dipelajari melalui berbagai referensi implementasi sistem keamanan *hybrid* yang telah ada sebelumnya. Seluruh hasil kajian tersebut kemudian digunakan sebagai basis utama dalam merancang arsitektur penelitian yang relevan.

3.2 Perancangan Sistem

Sistem ini akan memiliki tiga komponen utama, yaitu satu laptop peneliti dan dua *virtual machine*. Laptop peneliti berperan sebagai *user* yang melakukan aktivitas *reconnaissance* dan juga mengirim *traffic* normal. *Virtual machine* pertama berfungsi sebagai mesin target yang akan menerima berbagai *traffic* dari *user*. Sementara itu, *virtual machine* kedua berperan sebagai mesin IPS yang ditempatkan di antara *user* dan target. Topologi ini memastikan bahwa setiap paket yang dikirim dari *user* ke target dan sebaliknya harus melewati IPS sehingga dapat dianalisis dan dicatat. Keseluruhan langkah penelitian ini akan dijalankan berdasarkan topologi pada Gambar 3.2.



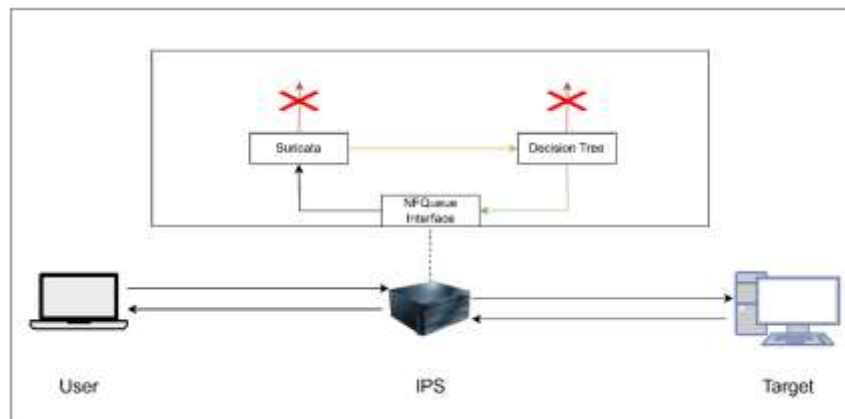
Gambar 3.2 Topologi lingkungan penelitian

3.2.1 Perancangan Lingkungan Penelitian

User akan berjalan menggunakan sistem operasi Kali Linux. Pemilihan Kali Linux didasarkan pada ketersediaan beragam aplikasi yang mendukung aktivitas *reconnaissance*. Hal ini dapat memudahkan penyusunan berbagai skenario pemindaian pada tahap pengumpulan data. Berbagai aplikasi *reconnaissance* seperti Nmap, Unicornscan, Hping3, dan Masscan akan dipasang pada mesin ini. Pengecekan versi dari setiap aplikasi akan dilakukan untuk memverifikasi bahwa proses instalasi berjalan dengan lancar.

User tidak hanya akan mengirimkan *traffic reconnaissance*, tetapi juga mengirim *traffic* normal dengan konfigurasi IP yang berbeda. Perbedaan IP antara *traffic reconnaissance* dan *traffic* normal dirancang agar mempermudah proses analisis hasil pengujian. Dengan begitu, setiap kategori *traffic* dapat diidentifikasi pada mesin IPS berdasarkan IP sumbernya.

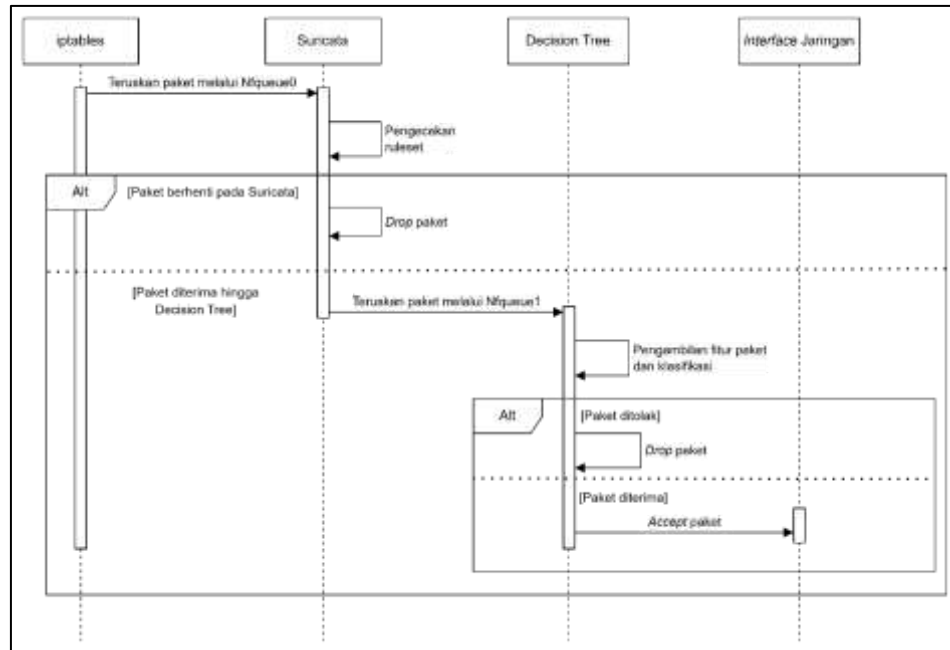
Mesin IPS akan dikonfigurasi sebagai *gateway* bagi mesin target sehingga seluruh rute jaringan diarahkan melalui mesin ini. Skema IP diatur sehingga *user* hanya dapat menjangkau target melalui alamat IP mesin IPS. Aturan *routing* pada masing-masing mesin juga disesuaikan untuk mendukung pola *traffic* ini. Topologi ini memungkinkan identifikasi aktivitas *reconnaissance* pada IPS sebelum paket mencapai target.



Gambar 3.3 Topologi dengan komponen di dalam mesin IPS

Gambar 3.3 mengilustrasikan mekanisme *hybrid* IPS yang terletak pada mesin IPS. Suricata akan bertindak sebagai lapisan pertama. Paket yang terdeteksi sebagai aktivitas *reconnaissance* akan langsung diblokir sebagaimana ditandai dengan tanda panah merah. Paket yang lolos dari pemeriksaan Suricata direpresentasikan oleh tanda panah kuning. Paket tersebut akan diteruskan ke komponen kedua, yakni model Decision Tree. Validitas paket kembali diperiksa pada komponen ini. Jika model Decision Tree mendeteksi aktivitas *reconnaissance* maka paket akan dibuang sesuai tanda panah merah kedua. Sebaliknya, tanda panah hijau menunjukkan paket yang telah diidentifikasi aman oleh kedua komponen dan diterima.

Alur paket pada mesin IPS secara rinci divisualisasikan pada Gambar 3.4. Alur paket ini dimulai dari iptables yang akan meneruskan paket yang tiba pada mesin IPS ke Nfqueue 0. Dengan begitu, setiap paket yang menuju ke mesin target terlebih dahulu dimasukkan ke antrean Nfqueue 0. Suricata akan menganalisis paket pada Nfqueue 0 berdasarkan *ruleset* yang telah dirancang untuk memblokir aktivitas *reconnaissance*. Mulai dari tahapan ini, alur paket akan berbeda sesuai dengan hasil Suricata. Jika Suricata menemukan *signature* paket yang sesuai dengan *ruleset*, maka Suricata akan memberikan keputusan *drop*. Jika paket tidak sesuai dengan *signature* yang ada pada *ruleset*, maka akan diteruskan pada komponen Decision Tree pada Nfqueue 1. Sebuah kode program akan mengambil fitur pada paket tersebut dan melakukan klasifikasi menggunakan model Decision Tree. Kode program berbasis Python akan memberikan keputusan *accept* jika Decision Tree mengidentifikasi paket sebagai kategori normal. Kemudian paket diteruskan menuju mesin target. Sebaliknya, jika Decision Tree mengidentifikasi sebagai aktivitas *reconnaissance* maka kode program memberikan keputusan *drop*. Lingkungan penelitian ini memungkinkan pengumpulan data dan pengujian skenario *reconnaissance* secara terkontrol.



Gambar 3.4 Alur paket pada mesin IPS

Mesin target akan dirancang sebagai server yang menjalankan berbagai layanan jaringan pada sejumlah *port* yang berbeda. Layanan yang akan dipasang berupa layanan web, email, *secure shell* (SSH), *file transfer protocol* (FTP), Microsoft *Directory Service*, dan berbagai jenis *database*. Seluruh layanan akan dijalankan menggunakan Docker untuk mempermudah proses instalasi. Seluruh *port* akan dibuka untuk mempermudah pemindaian. Pembukaan koneksi ke berbagai *port* bertujuan agar aktivitas *reconnaissance* yang dilakukan oleh *user* dapat mengidentifikasi layanan yang berjalan pada target. Pola pemindaian yang akan disimpan ini mencerminkan interaksi nyata antara alat pemindaian dan layanan jaringan. Dengan demikian, data yang dihasilkan dari proses pemindaian dapat dimanfaatkan secara optimal pada tahap perancangan *ruleset* Suricata dan pelatihan model Decision Tree.

3.2.2 Perancangan Pengumpulan Data

Tahap pengumpulan data difokuskan untuk menyediakan data untuk mendukung seluruh rangkaian penelitian. Data yang dikumpulkan diklasifikasikan menjadi dua kategori, yakni *traffic* aktivitas *reconnaissance* dan *traffic* normal. Kategori *reconnaissance* merepresentasikan indikasi serangan, sedangkan kategori normal mencerminkan kondisi operasional jaringan yang wajar. Pengumpulan kedua kategori *traffic* akan dilakukan menggunakan bantuan *script* dan kode program yang telah disediakan dalam kerangka kerja AB-TRAP milik Bertoli dkk., (2021).

Kedua jenis *traffic* tersebut akan melalui serangkaian pemrosesan agar dapat dimanfaatkan secara optimal dalam analisis. Data dari beberapa aktivitas *reconnaissance* digunakan untuk mengidentifikasi karakteristik pola serangan. Hasil identifikasi tersebut menjadi landasan krusial dalam

perancangan *ruleset* yang akan diterapkan pada Suricata. Selain itu, kedua kategori data akan disusun menjadi sebuah *dataset* terstruktur guna melatih model Decision Tree.

Pengumpulan data diawali dengan pengambilan *traffic* aktivitas *reconnaissance* yang dihasilkan dari *user* menuju target. *User* akan dikonfigurasi dengan IP penyerang sebelum melakukan *reconnaissance*. Aktivitas *reconnaissance* dijalankan menggunakan aplikasi pemindaian Nmap, Unicornscan, dan Hping3 terhadap target. Seluruh *traffic* yang melewati mesin IPS selama proses pemindaian akan disimpan dalam format berkas pcap. Pemilihan mesin IPS sebagai pengumpul data didasari oleh rencana implementasi sistem *hybrid* IPS pada mesin tersebut. Pendekatan ini bertujuan menyelaraskan kondisi pengambilan data dengan implementasi sistem yang sebenarnya. Detail teknik pemindaian yang akan disimpan pada tahap ini terdapat pada Tabel 3.1.

Tabel 3.1 Teknik pemindaian pada tahap pengumpulan data

No	Teknik Pemindaian
1	Nmap TCP SYN Scan
2	Nmap TCP Connect Scan
3	Nmap TCP NULL Scan
4	Nmap TCP XMAS Scan
5	Nmap TCP FIN Scan
6	Nmap TCP ACK Scan
7	Nmap TCP Window Scan
8	Nmap TCP Maimon Scan
9	Nmap TCP Connect Scan Random Data
10	Unicornscan TCP SYN Scan
11	Unicornscan TCP Connect Scan
12	Unicornscan TCP NULL Scan
13	Unicornscan TCP XMAS Scan
14	Unicornscan TCP FIN Scan
15	Unicornscan TCP ACK Scan
16	Hping3 TCP SYN Scan
17	Hping3 TCP NULL Scan
18	Hping3 TCP XMAS Scan
19	Hping3 TCP FIN Scan
20	Hping3 TCP ACK Scan
21	Masscan TCP Scan

Tahap pengumpulan data akan dilanjutkan dengan pembuatan *dataset traffic reconnaissance* untuk melatih model Decision Tree. Data latih hanya akan menggunakan teknik pemindaian yang tercantum pada Tabel 3.2. Pendekatan ini menempatkan teknik pemindaian tersebut sebagai

representasi aktivitas *reconnaissance* yang telah diketahui. Sebaliknya, teknik pemindaian lainnya akan diasumsikan sebagai serangan *zero-day* atau varian *reconnaissance* baru. Berkas pcap hasil pemindaian akan dikonversi menjadi format CSV melalui proses ekstraksi fitur. Proses ini akan mengambil nilai fitur dari lapisan *data link*, *network*, dan *transport* pada setiap paket. Selain itu, ditambahkan juga data label sesuai dengan teknik pemindaian yang dilakukan. Berkas CSV ini nantinya akan digabungkan dengan *traffic* normal untuk melatih model.

Tabel 3.2 Teknik pemindaian pada dataset latih

No	Teknik Pemindaian
1	Nmap TCP SYN Scan
2	Nmap TCP Connect Scan
3	Nmap TCP NULL Scan
4	Nmap TCP FIN Scan
5	Nmap TCP Maimon Scan
6	Unicornscaan TCP XMAS Scan
7	Unicornscaan TCP ACK Scan
8	Hping3 TCP NULL Scan
9	Hping3 TCP FIN Scan

Traffic dari dataset MAWILab dipilih sebagai representasi *traffic* normal pada penelitian ini. Menurut Fontugne dkk., (2010) dalam Schraven dkk., (2025), MAWILab merupakan *dataset* berisikan *traffic* jaringan nyata dari jalur *backbone* antara Jepang dan Amerika Serikat. *Dataset* ini menyediakan data harian dalam format pcap yang mencakup berbagai aktivitas *traffic* normal maupun anomali. Seluruh koleksi *traffic* data tersebut tersedia dan dapat diunduh secara terbuka melalui laman web resmi proyek MAWILab. Pemilihan MAWILab didasari oleh karakteristiknya sebagai sumber *traffic* asli yang mampu merepresentasikan kondisi jaringan sesungguhnya. Terdapat 2 jenis data yang dapat diunduh melalui situs resmi MAWILab sesuai pada Tabel 3.3.

Tabel 3.3 Jenis data pada dataset MAWILab

No	Jenis Data	Keterangan	Format
1	<i>Traffic</i> jaringan	Data hasil Tcpcdump berisi <i>traffic</i> mentah, dengan <i>payload</i> dan IP dianonimkan untuk menjaga privasi.	pcap
2	Klasifikasi <i>traffic</i>	Berisi <i>metadata</i> seperti IP, <i>port</i> , dan hasil klasifikasi dari <i>traffic</i> . Data ini berfungsi	XML/CSV

		sebagai sumber kebenaran dari klasifikasi <i>traffic</i> anomali atau normal.	
--	--	---	--

Penelitian ini akan menggunakan data *traffic* terbaru yang telah disediakan oleh MAWILab. Langkah pertama adalah konversi data klasifikasi berformat CSV agar dapat digunakan oleh Tshark. Sebuah kode program akan digunakan untuk membaca dan memproses setiap baris data tersebut. Hasil konversi kemudian disimpan sebagai *syntax* filter Tshark dalam format teks. Berkas *syntax* ini akan menjadi komponen krusial sebelum pemrosesan berkas pcap dilakukan.

Tantangan selanjutnya adalah ukuran berkas pcap dari MAWILab yang sangat besar. Oleh karena itu, pcap akan dipecah menjadi segmen-segmen kecil berisi 1 juta paket. Segmentasi ini bertujuan agar proses filtrasi memungkinkan untuk dilakukan dan meringankan beban komputasi. Proses pemecahan dan penyaringan paket tersebut akan dilakukan secara otomatis menggunakan beberapa *script*. *Script* ini hanya akan menyimpan paket normal berprotokol TCP dan menghapus *traffic* yang ditandai sebagai *anomalous* atau *malicious*. Dengan demikian, *traffic* normal akan tersusun dari paket TCP yang telah dibersihkan dari indikasi serangan.

Pengambilan sampel dilakukan pada *traffic* normal untuk membatasi jumlah data yang digunakan. Langkah ini bertujuan menyeimbangkan proporsi antara *traffic reconnaissance* dan *traffic* normal. Sebanyak 55 ribu paket awal dipilih sebagai data latih dan sekitar 55 ribu paket akhir menjadi data uji. Pemisahan ini didasari untuk menghindari adanya data yang sama antara data latih dan data uji. Data latih akan melalui proses ekstraksi fitur yang sama seperti pada data *traffic reconnaissance* dan disimpan dalam format CSV.

3.2.3 Perancangan Ruleset Suricata

Ruleset Suricata pada penelitian ini dirancang dengan tujuan memblokir *traffic reconnaissance*. Tahap awal penelitian berfokus pada analisis karakteristik dari berbagai teknik pemindaian pada Tabel 3.2. Pendekatan Liao dkk. (2020) diadopsi untuk mengidentifikasi anomali berdasarkan *header* TCP. Fitur yang digunakan contohnya *flags*, *window*, *Maximum Segment Size* (MSS), dll. Hasil analisis ini bertujuan menemukan *signature* yang menjadi ciri *traffic reconnaissance*.

Signature yang ditemukan selanjutnya akan diterjemahkan menjadi *ruleset* dalam format *syntax* Suricata. Kata kunci pada Tabel 3.4 digunakan untuk mencocokkan *signature* pada paket yang masuk. Setiap *ruleset* akan dilengkapi dengan *Signature* ID (SID) unik untuk mempermudah identifikasi jenis serangan yang terdeteksi. Keputusan *drop* akan diterapkan pada setiap *ruleset* untuk memblokir paket *reconnaissance* secara langsung. Kumpulan

ruleset ini kemudian dikumpulkan menjadi sebuah berkas *ruleset* utama yang akan dimuat oleh sistem.

Tabel 3.4 Kata kunci pada *ruleset* Suricata

No	Kata Kunci	Keterangan
1	tcp.flags	Kata kunci ini berfungsi untuk mencocokkan kombinasi bit <i>flags</i> pada <i>header</i> protokol TCP.
2	window	Kata kunci ini digunakan untuk memeriksa nilai spesifik ukuran <i>window</i> paket TCP
3	tcp.wscale	Kata kunci ini mendeteksi nilai MSS yang disisipkan dalam opsi paket TCP
4	tcp.mss	Kata kunci ini berfungsi untuk memvalidasi nilai <i>window scale</i> yang terdapat dalam opsi <i>header</i> TCP

Suricata akan dikonfigurasi untuk beroperasi dalam mode IPS dengan memanfaatkan mekanisme Nfqueue. Sistem diatur untuk memantau *traffic* yang masuk melalui antrean Nfqueue 0. Paket yang cocok dengan *ruleset* akan langsung dibuang, sedangkan paket yang tidak terdeteksi diteruskan ke Nfqueue 1. Seluruh aktivitas deteksi dan pemblokiran dicatat secara otomatis ke dalam berkas *logging*. Informasi ringkas disimpan dalam *fast log*, sementara *metadata* lengkap disimpan dalam format JSON melalui *Extensible Event Format* (EVE) *log*. Aturan konfigurasi khusus juga diterapkan sehingga Suricata tidak memblokir paket di luar *ruleset*. Hal ini penting dikarenakan *traffic* normal hasil sampling MAWILab tidak membentuk *flow* yang jelas. *Flow* yang tidak jelas ini secara *default* akan diblokir oleh Suricata jika tidak diatur pada konfigurasinya. Setelah perancangan Suricata selesai, tahap selanjutnya berfokus pada pelatihan model Decision Tree sebagai lapisan pertahanan kedua.

3.2.4 Perancangan Model Decision Tree

Model Decision Tree akan diterapkan sebagai lapisan pertahanan kedua guna memeriksa paket yang lolos dari Suricata. Implementasi ini bertujuan mendeteksi varian *reconnaissance* baru yang mungkin gagal ditangkap oleh *ruleset* Suricata. Namun, model Decision Tree harus melalui proses pelatihan agar dapat bekerja dengan efektif.

Perancangan model Decision Tree akan diawali dengan pemrosesan berkas CSV hasil ekstraksi fitur. Proses ini akan dilaksanakan dengan mengikuti alur kerja pada kerangka kerja AB-TRAP. Fokus utamanya adalah menyeleksi

fitur relevan yang dapat membedakan lalu lintas normal dan *reconnaissance*. Langkah ini bertujuan untuk mendapatkan atribut yang memberikan pemisah paling jelas antar kelas data.

Data *traffic* normal akan dilengkapi dengan kolom label kelas yang belum tersedia sebelumnya. Atribut ini berfungsi sebagai acuan bagi Decision Tree untuk mengidentifikasi kategori normal atau *reconnaissance*. Data CSV dari *traffic* normal dan *reconnaissance* akan digabungkan terlebih dahulu sebelum pemrosesan dimulai. Selanjutnya, seluruh nilai *NULL* yang ditemukan pada *dataset* akan dikonversi menjadi angka 0. Hal ini penting dilakukan karena model tidak dapat memproses atau mengubah nilai *NULL* secara otomatis. Pemrosesan dilanjutkan dengan penyaringan terhadap paket dengan protokol selain TCP.

Pemrosesan data akan dilanjutkan dengan mengeliminasi fitur yang terlalu spesifik terhadap lingkungan uji coba. Informasi tersebut mencakup data pada layer *data link*, TTL, IP, serta nomor *port*. Selain itu, atribut dengan nilai acak seperti nomor urut seperti *sequence*, *acknowledgement*, dan *checksum* juga akan dihapus. Penghapusan fitur ini bertujuan meningkatkan kemampuan generalisasi model saat diterapkan pada lingkungan jaringan berbeda. Fitur tanpa variasi nilai juga akan dihapus karena tidak memberikan kontribusi pada proses pembelajaran model. Kemudian label kelas pada data akan diubah dengan nilai biner (0 dan 1). Format ini membatasi fokus pembelajaran pada dua kategori utama, yakni normal dan *reconnaissance*.

Tahapan pemrosesan data akan diakhiri dengan penghapusan data duplikat dan penghapusan fitur IP ID. Penghapusan duplikat akan dilakukan terlebih dahulu dengan tetap mempertahankan keberadaan kolom IP ID. Strategi ini diterapkan untuk mencegah penyusutan data secara masif jika IP ID dihapus lebih awal. Pendekatan tersebut juga berfungsi sebagai mekanisme *oversampling* guna menyeimbangkan proporsi kelas data. Kemudian dataset yang telah bersih akan disimpan dalam format CSV untuk keperluan pelatihan dan *deployment* model.

Transformasi data akan dilakukan setelah proses seleksi dan pembersihan fitur selesai dilaksanakan. Seluruh fitur akan diproses menggunakan metode *scaling* dengan menerapkan teknik standarisasi data. Proses ini bertujuan menyasrakan rentang nilai antar fitur agar distribusi data menjadi seragam.

Eksperimen *hyperparameter tuning* akan dijalankan untuk mencari konfigurasi optimal model Decision Tree. Pengujian akan mencakup cara dan perhitungan pemisahan, hingga kedalaman pohon. Proses evaluasi akan dirancang menggunakan mekanisme perulangan bersarang. Iterasi utama akan dilakukan sebanyak 10 kali dengan menerapkan teknik *cross validation* sebanyak 5 kali. Pada setiap iterasi, kinerja kombinasi *hyperparameter* akan diukur berdasarkan hasil *f1-score*. Hasil akhir pengujian ini berupa 10 kandidat konfigurasi terbaik dari setiap iterasi. Konfigurasi final akan dipilih berdasarkan kombinasi *hyperparameter* yang paling sering muncul.

Model Decision Tree akan dilatih kembali menggunakan konfigurasi *hyperparameter* yang telah terbukti optimal. Model dan objek standardisasi akan disimpan ke dalam berkas dengan format yang dapat dibaca oleh kode program pada mesin IPS. Proses penyimpanan ini dilakukan agar meniadakan beban komputasi untuk pelatihan ulang setiap kali sistem dijalankan. Kode program akan dirancang untuk membaca paket yang diteruskan oleh Suricata, mengekstrak fitur relevan dan menerapkan *scaling* sebelum melakukan klasifikasi. Hasil klasifikasi kemudian akan diterjemahkan menjadi keputusan untuk menerima atau memblokir paket. Mekanisme ini memungkinkan sistem IPS melakukan pemblokiran berbasis ML yang tidak terdeteksi oleh *ruleset* Suricata.

3.3 Skenario Pengujian dan Evaluasi

Skenario pengujian akan dilakukan setelah setiap komponen pada sistem *hybrid* IPS berjalan dengan baik. Pengujian akan dilakukan dengan mengirim *traffic* normal dan *reconnaissance* dari *user*. Data *traffic* normal yang akan diujikan adalah data yang tidak pernah digunakan pada tahap pelatihan. Data *traffic* normal akan menggunakan data hasil sampling sebanyak 55 ribu paket yang diperoleh dari tahap pengumpulan data. *Traffic* normal akan dikirim pada target menggunakan sebuah kode program khusus. Kode program ini akan mengganti IP sumber dan tujuan kemudian mengirimkan paket-paket tersebut ke target. Pengujian aktivitas *reconnaissance* akan menggunakan semua teknik pemindaian pada Tabel 3.1. Dengan begitu, kemampuan sistem dalam mendeteksi baik serangan yang telah diketahui maupun serangan jenis baru dapat diuji. Tabel 3.5 merinci teknik pemindaian yang tidak terdapat pada data latih dan akan digunakan pada pengujian.

Tabel 3.5 Teknik pemindaian di luar data latih

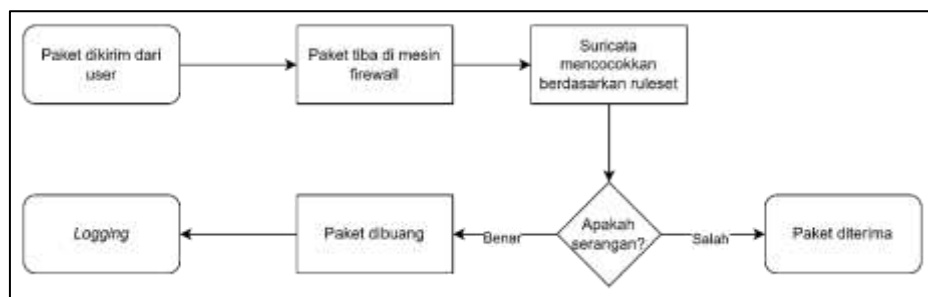
No	Teknik Pemindaian
1	Nmap TCP XMAS Scan
2	Nmap TCP ACK Scan
3	Nmap TCP Window Scan
4	Nmap TCP ACK Scan
5	Nmap TCP Connect Scan Dengan Data Random
6	Unicornsweep TCP SYN Scan
7	Unicornsweep TCP Connect Scan
8	Unicornsweep TCP NULL Scan
9	Unicornsweep TCP FIN Scan
10	Hping3 TCP SYN Scan
11	Hping3 TCP XMAS Scan
12	Masscan TCP scan

Evaluasi hasil pengujian akan dilakukan dengan menghitung jumlah paket yang berhasil diterima oleh target. Selain itu, data paket yang terblokir akan

divalidasi menggunakan berkas *logging* dari Suricata dan model Decision Tree. *Traffic* normal dan *reconnaissance* dikirim dari *user* dengan menggunakan alamat IP sumber yang berbeda. Alamat IP sumber ini berperan sebagai acuan kebenaran untuk menilai apakah paket diperlakukan sesuai dengan kategorinya oleh IPS. Paket yang berasal dari IP yang ditetapkan sebagai pengirim *traffic* normal diharapkan selalu diterima oleh sistem. Sebaliknya, paket yang berasal dari IP yang ditetapkan sebagai pengirim *traffic reconnaissance* diharapkan ditolak oleh IPS. Dengan pendekatan ini, proses verifikasi keberhasilan pemisahan *traffic* normal dan *reconnaissance* menjadi lebih terukur dan mudah dianalisis.

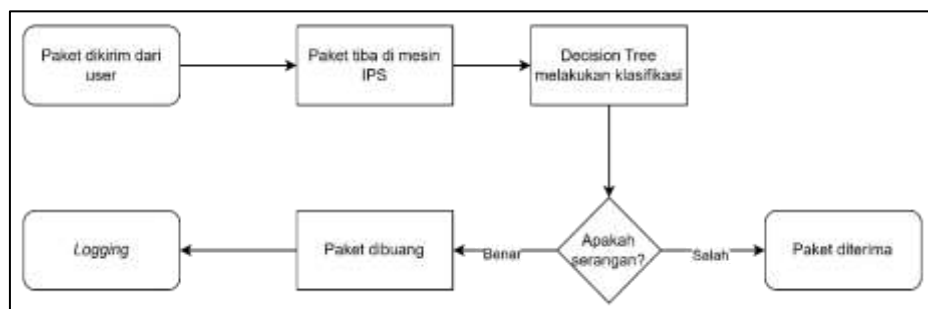
Skenario pengujian akan dilakukan dengan membandingkan tiga pendekatan IPS, yakni:

1. Suricata tunggal: skenario pengujian pertama akan membatasi fungsi sistem dengan hanya mengaktifkan komponen Suricata. Setiap paket yang tiba akan dipindai berdasarkan *ruleset* yang telah disusun sebelumnya. Jika tidak ditemukan kecocokan *signature*, paket tersebut akan diteruskan langsung menuju target. Ilustrasi lengkap mengenai alur pada skenario ini dapat dilihat pada Gambar 3.5.



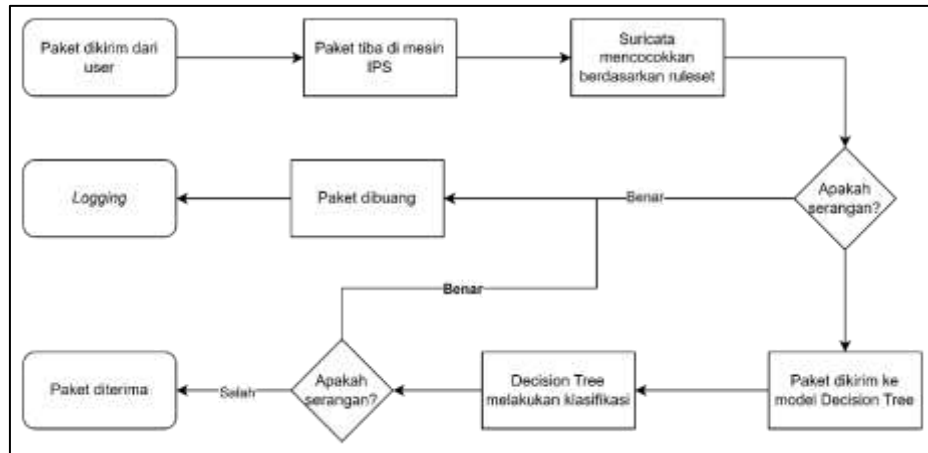
Gambar 3.5 Alur paket pada skenario Suricata tunggal

2. Decision Tree tunggal: skenario kedua akan memfokuskan pengujian pada kinerja klasifikasi model Decision Tree. Setiap paket data yang tiba akan langsung diproses oleh model untuk melakukan klasifikasi. Paket hanya akan diteruskan menuju target apabila model mengklasifikasikan sebagai *traffic* normal. Mekanisme ini bertujuan mengukur efektivitas model Decision Tree tanpa bantuan deteksi Suricata. Ilustrasi lengkap mengenai alur pada skenario ini dapat dilihat pada Gambar 3.6.



Gambar 3.6 Alur paket pada skenario Decision Tree tunggal

3. *Hybrid IPS*: Skenario pengujian terakhir akan mengintegrasikan Suricata dan model Decision Tree dalam satu sistem *hybrid IPS*. Proses inspeksi paket akan diawali dengan pemindaian *signature* pada *ruleset* Suricata. Paket yang tidak memicu deteksi Suricata akan diteruskan menuju model Decision Tree. Tahap selanjutnya melibatkan klasifikasi oleh model untuk verifikasi lanjutan pada paket. Paket hanya akan diteruskan apabila model mengklasifikasikan sebagai *traffic normal*. Ilustrasi lengkap mengenai alur pada skenario ini dapat dilihat pada Gambar 3.7.



Gambar 3.7 Alur paket pada Skenario *hybrid IPS*

Ketiga skenario akan diuji menggunakan sumber *traffic* yang sama demi menjaga objektivitas. Hasil akan direkapitulasi ke dalam *confusion matrix* yang memuat nilai *true positive*, *true negative*, *false positive*, dan *false negative*. *Matrix* ini berfungsi menggambarkan kemampuan sistem dalam membedakan antara *traffic normal* dan *traffic reconnaissance*. Keterangan dari setiap nilai pada *confusion matrix* dijelaskan pada Tabel 3.6. Nilai tersebut kemudian diturunkan menjadi metrik evaluasi seperti akurasi, presisi, *recall*, serta *f1-score*. Dengan demikian, analisis tidak hanya berfokus pada kuantitas serangan yang berhasil dideteksi. Evaluasi juga meninjau potensi gangguan *traffic normal* akibat kenaikan *false positive*.

Tabel 3.6 Keterangan nilai *confusion matrix*

Aktual/Prediksi	Reconnaissance	Normal
Reconnaissance	Paket <i>reconnaissance</i> yang terblokir oleh IPS	Paket <i>reconnaissance</i> yang sampai ke target
Normal	Paket normal yang terblokir oleh IPS	Paket normal yang sampai ke target

Hasil dari setiap skenario pengujian akan dianalisis guna mengevaluasi efektivitas mekanisme pemblokiran. Analisis tersebut akan mengidentifikasi secara spesifik teknik pemindaian yang berhasil lolos dari deteksi setiap skenario. Temuan

ini akan memberikan wawasan tambahan sebagai bahan pertimbangan krusial terkait implementasi komponen sesuai skenario.

Penelitian juga ini akan mengukur *throughput* untuk mengevaluasi dampak setiap pendekatan terhadap kinerja jaringan. *Throughput* merepresentasikan kemampuan jaringan mempertahankan laju transfer data saat paket melewati proses inspeksi. Metrik ini dapat menunjukkan *overhead* yang ditimbulkan oleh Suricata, Decision Tree, maupun kombinasi keduanya. Pengukuran dilakukan dalam empat skenario yakni *baseline* (tanpa komponen), Suricata tunggal, Decision Tree tunggal, dan sistem *hybrid*. Parameter pengujian akan dibuat konsisten agar dapat terlihat perbedaan setiap skenarionya. Pengujian *throughput* akan dilaksanakan menggunakan aplikasi iperf3. Setiap skenario dijalankan selama 100 detik dan diulang sebanyak 10 kali untuk memastikan konsistensi data. Diantara setiap iterasi akan dilakukan jeda selama 1 menit. Waktu jeda ini diperlukan untuk memastikan *buffer* memori bersih dan kondisi jaringan kembali stabil. Hasil pengukuran kemudian dibandingkan untuk menilai korelasi antara peningkatan kemampuan deteksi dan penurunan kinerja jaringan. Temuan ini selanjutnya digunakan sebagai dasar pembahasan mendalam pada bab berikutnya.

3.4 Pengambilan Kesimpulan

Tahap terakhir dari penelitian ini adalah penarikan kesimpulan. Kesimpulan akan dirumuskan berdasarkan analisis mendalam terhadap temuan-temuan yang diperoleh dari tahap pengujian. Analisis ini secara khusus akan membandingkan efektivitas antara komponen Suricata, Decision Tree, atau keduanya dalam mengidentifikasi aktivitas jaringan normal dan aktivitas *reconnaissance*. Hasil rangkuman akan menjelaskan jawaban dari rumusan masalah dan saran untuk penelitian lanjutan dari topik ini.

BAB 4 IMPLEMENTASI

4.1 Implementasi Lingkungan Penelitian

Implementasi lingkungan penelitian diawali dengan tahap konfigurasi mesin *user*. *User* beroperasi menggunakan sistem operasi Kali Linux guna menjalankan aktivitas *reconnaissance* serta mengirimkan *traffic* normal. Spesifikasi sistem operasi yang digunakan tertera pada Gambar 4.1.

```
$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2025.3"
VERSION="2025.3"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"
```

Gambar 4.1 Versi Kali Linux pada mesin *user*

Tahap selanjutnya melibatkan proses instalasi aplikasi untuk menjalankan aktivitas *reconnaissance*. Proses instalasi aplikasi dieksekusi melalui baris perintah terminal pada sistem operasi Kali Linux. Instruksi tersebut mencakup perintah instalasi untuk aplikasi Nmap, Unicornscan, Hping3, serta Masscan. Verifikasi versi pada setiap aplikasi dilakukan setelah proses instalasi berhasil diselesaikan. Gambar 4.2 merupakan dokumentasi verifikasi aplikasi *reconnaissance*.

```
$ sudo nmap --version
Nmap version 7.95 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.7 openssl-3.5.2 libssh2-1.11.1 libz-1.3.1 libpcap-1.10.5
nmap-libidnet-1.12 ipv6
Compiled without:
Available nsock engines: epoll poll select

$ sudo unicornscan --version
unicornscan version '0.4.7' using module version 1.03 build options [ ]
pcap version libpcap version 1.10.5 (with TPACKET_V3)
Compiled by kalibuild on Linux dionysus 4.19.0-26-amd64 x86_64 at Tue May 7 01:46:59 UTC 2024 wit
h gcc version 13.2.013.2.024
report bugs to esace-users@lists.sourceforge.net

$ sudo hping3 --version
hping3 version 3.0.0-alpha-2 ($Id: release.h,v 1.4 2004/04/09 23:38:56 antirez Exp $)
This binary is TCL scripting capable

$ sudo masscan --version
Masscan version 1.3.2 ( https://github.com/robertdavidgraham/masscan )
Compiled on: Jul 25 2024 16:44:22
Compiler: gcc 10.1.0
OS: Linux
CPU: unknown (64 bits)
GIT version: unknown
```

Gambar 4.2 Versi aplikasi *reconnaissance*

Tahapan akhir konfigurasi pada mesin *user* meliputi konfigurasi IP beserta *routing*. Penerapan konfigurasi IP tersebut bergantung pada jenis *traffic* yang

sedang dikirimkan. Pengiriman *traffic* dilakukan pada *interface* jaringan yang sama, namun IP yang berbeda untuk membedakan *traffic reconnaissance* dan normal. *Routing* juga disesuaikan agar seluruh *traffic* menuju target melalui mesin IPS. Konfigurasi IP serta *routing* yang diterapkan terdapat pada Gambar 4.3.

```

ip a show eth6
13: eth6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1478 qdisc mq state UP group default qlen 1000
    link/ether 08:15:5d:92:5d:8b brd ff:ff:ff:ff:ff:ff
    inet 192.168.200.3/32 brd 192.168.200.3 scope global noprefixroute eth6
        valid_lft forever preferred_lft forever

ip a show eth6
14: eth6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1420 qdisc mq state UP group default qlen 1000
    link/ether 08:15:5d:26:fb:3f brd ff:ff:ff:ff:ff:ff
    inet 192.168.200.4/32 brd 192.168.200.4 scope global noprefixroute eth6
        valid_lft forever preferred_lft forever

ip route show 192.168.200.8/24
192.168.200.0/24 dev eth6 proto kernel scope link metric 5

```

Gambar 4.3 Konfigurasi IP dan *routing* pada user

Penerapan mekanisme pada mesin IPS dibantu menggunakan *script* pada Tabel 4.1. Variabel `TARGET_IP` didefinisikan sehingga iptables secara spesifik mengarahkan paket yang hanya menuju target menuju Nfqueue 0. Aturan iptables diterapkan pada *chain* `PREROUTING`, sehingga paket dapat diarahkan ke *userspace* untuk dianalisis oleh Suricata sebelum diteruskan ke tujuannya. Opsi `-q 0` digunakan agar Suricata memproses paket data yang masuk melalui antrean Nfqueue 0.

Tabel 4.1 Script *hybrid IPS*

Turn On hybrid IPS	
1	<code>#!/usr/bin/env bash</code>
2	<code>set -euo pipefail</code>
3	<code>TARGET_IP=192.168.200.2/32</code>
4	
5	<code>echo "[*] Menerapkan iptables rules"</code>
6	
7	<code>iptables -t mangle -D PREROUTING -i wg0</code>
8	<code>-d "\$TARGET_IP" -j NFQUEUE --queue-num 0 2>/dev/null</code>
9	<code> true</code>
10	
11	<code>iptables -t mangle -A PREROUTING -i wg0</code>
12	<code>-d "\$TARGET_IP" -j NFQUEUE --queue-num 0</code>
13	
14	<code>systemctl stop suricata true</code>
15	<code>suricata -D -c /etc/suricata/suricata-recon.yaml -q 0</code>
16	
17	<code>echo "[*] Suricata RECON berjalan @Q0."</code>
18	<code>echo "[*] Hybrid FW aktif. ML harus bind ke Q1."</code>

Pada mesin target terdapat berbagai layanan seperti web, email, *secure shell* (SSH), *file transfer protocol* (FTP), Microsoft *Directory Service*, dan berbagai jenis *database*. Kotak merah pada Gambar 4.4 menunjukkan layanan yang sudah berjalan. Setiap layanan berjalan pada *port default* yang umum digunakan oleh layanan tersebut.

```
LISTEN 0 4096 0.0.0.0:pop3 0.0.0.0:* users:(("docker-proxy",pid=6126,fid=7))
LISTEN 0 4096 0.0.0.0:imap 0.0.0.0:* users:(("docker-proxy",pid=6100,fid=7))
LISTEN 0 4096 0.0.0.0:microsoft-ds 0.0.0.0:* users:(("docker-proxy",pid=6591,fid=7))
LISTEN 0 4096 0.0.0.0:https 0.0.0.0:* users:(("docker-proxy",pid=6753,fid=7))
LISTEN 0 4096 0.0.0.0:http 0.0.0.0:* users:(("docker-proxy",pid=7745,fid=7))
LISTEN 0 4096 0.0.0.0:pop3 0.0.0.0:* users:(("docker-proxy",pid=5857,fid=7))
LISTEN 0 4096 0.0.0.0:smtp 0.0.0.0:* users:(("docker-proxy",pid=5922,fid=7))
LISTEN 0 4096 0.0.0.0:ssh 0.0.0.0:* users:(("docker-proxy",pid=5673,fid=7))
LISTEN 0 4096 0.0.0.0:ftp 0.0.0.0:* users:(("docker-proxy",pid=5681,fid=7))
LISTEN 0 4096 0.0.0.0:ftp-data 0.0.0.0:* users:(("docker-proxy",pid=6485,fid=7))
LISTEN 0 4096 0.0.0.0:imap2 0.0.0.0:* users:(("docker-proxy",pid=5871,fid=7))
LISTEN 0 4096 0.0.0.0:mysql-s 0.0.0.0:* users:(("docker-proxy",pid=6687,fid=7))
LISTEN 0 4096 0.0.0.0:mysql 0.0.0.0:* users:(("docker-proxy",pid=6659,fid=7))
LISTEN 0 4096 0.0.0.0:postgresql 0.0.0.0:* users:(("docker-proxy",pid=5161,fid=7))
LISTEN 0 4096 0.0.0.0:redis 0.0.0.0:* users:(("docker-proxy",pid=4488,fid=7))
```

Gambar 4.4 Layanan yang berjalan pada target

4.2 Implementasi Pengumpulan Data

Pada sub bab ini, dijelaskan proses pengumpulan data yang terbagi menjadi 2 yakni *traffic normal* dan *traffic reconnaissance*.

4.2.1 Traffic Reconnaissance

Implementasi pengumpulan data *reconnaissance* dimulai pada mesin IPS. Kode program pada mesin IPS bersiap menerima paket UDP pada *port* 12000 untuk instruksi kontrol penyimpanan *traffic*. Kode program akan menginisiasi eksekusi Tcpcdump ketika menerima paket UDP dengan data numerik. *Traffic* disimpan dengan format penamaan yang mengacu pada angka identifikasi tersebut. Proses penyimpanan *traffic* berlangsung hingga kode program menerima paket UDP dengan data "STOP". Instruksi ini secara otomatis menghentikan proses Tcpcdump yang sedang berjalan. Kode yang digunakan dituliskan dalam bentuk *pseudocode* pada Tabel 4.2.

Tabel 4.2 *Pseudocode* UDP server

UDP Server	
1	PRINT ">> Server running..."
2	
3	WHILE True
4	LISTEN TO UDP message AND client_address AT PORT
5	12000
6	PRINT ">> Received message from: " + client_address
7	
8	SET datehour TO CURRENT DATETIME
9	SET filename TO message + "_" + datehour + ".pcap"
10	
11	START BACKGROUND PROCESS p WITH COMMAND:
12	tcpdump -i wg0 tcp and src <client_address>

13	-s0 -nn -w <filename>
14	
15	WAIT UNTIL message IS "STOP"
16	
17	TRY
18	PRINT ">> Stopped process: " + p
19	TERMINATE PROCESS p
20	CATCH ANY ERROR
21	PRINT ">> Error to terminate process"
22	END TRY
23	END WHILE

User dapat mulai mengirimkan *traffic reconnaissance* setelah mesin IPS siap menerima dan menyimpan paket. User harus mengirim paket UDP sebagai penanda awal dan akhir pada setiap sesi pemindaian. Proses eksekusi pada sisi *user* ini dilakukan oleh *script* yang tercantum dalam Tabel 4.3.

Tabel 4.3 Script pemindaian TCP

Scanning TCP	
1	#!/bin/bash
2	
3	echo "Starting scanning: "
4	date
5	
6	target=192.168.200.2
7	sniff=192.168.200.1
8	
9	hping_pkt_count=1000
10	attacker_interface="eth6"
11	repeat_unicornscan=3
12	
13	while IFS= read -r IP
14	do
15	IP_masscan="\${IP}/32"
16	
17	echo "\$IP"
18	echo -n "01" >/dev/udp/\${sniff}/12000
19	sleep 5
20	echo "> sudo nmap --privileged TCP SYN Scan (dst_ip:
21	\${IP}) "
22	
23	sudo nmap --privileged -sS -Pn -n \${IP}
24	echo -n "STOP" >/dev/udp/\${sniff}/12000
25	sleep 5
26	

```

27     echo -n "02" >/dev/udp/${sniff}/12000
28     sleep 5
29     echo "> sudo nmap --privileged TCP Connect Scan
30 (dst_ip: ${IP})"
31
32     sudo nmap --privileged -sT -Pn -n ${IP}
33     echo -n "STOP" >/dev/udp/${sniff}/12000
34     sleep 5
35
36     echo -n "03" >/dev/udp/${sniff}/12000
37     sleep 5
38     echo "> sudo nmap --privileged TCP NULL Scan
39 (dst_ip: ${IP})"
40
41     sudo nmap --privileged -sN -Pn -n ${IP}
42     echo -n "STOP" >/dev/udp/${sniff}/12000
43     sleep 5
44
45     echo -n "04" >/dev/udp/${sniff}/12000
46     sleep 5
47     echo "> sudo nmap --privileged TCP XMAS Scan
48 (dst_ip: ${IP})"
49
50     sudo nmap --privileged -sX -Pn -n ${IP}
51     echo -n "STOP" >/dev/udp/${sniff}/12000
52     sleep 5
53
54     echo -n "05" >/dev/udp/${sniff}/12000
55     sleep 5
56     echo "> sudo nmap --privileged TCP FIN Scan (dst_ip:
57 ${IP})"
58
59     sudo nmap --privileged -sF -Pn -n ${IP}
60     echo -n "STOP" >/dev/udp/${sniff}/12000
61     sleep 5
62
63     echo -n "06" >/dev/udp/${sniff}/12000
64     sleep 5
65     echo "> sudo nmap --privileged TCP ACK Scan (dst_ip:
66 ${IP})"
67
68     sudo nmap --privileged -sA -Pn -n ${IP}
69     echo -n "STOP" >/dev/udp/${sniff}/12000
70     sleep 5
71
72     echo -n "07" >/dev/udp/${sniff}/12000

```

```

73     sleep 5
74     echo "> sudo nmap --privileged TCP Window Scan
75 (dst_ip: ${IP})"
76
77     sudo nmap --privileged -sW -Pn -n ${IP}
78     echo -n "STOP" >/dev/udp/${sniff}/12000
79     sleep 5
80
81     echo -n "08" >/dev/udp/${sniff}/12000
82     sleep 5
83     echo "> sudo nmap --privileged TCP Maimon Scan
84 (dst_ip: ${IP})"
85
86     sudo nmap --privileged -sM -Pn -n ${IP}
87     echo -n "STOP" >/dev/udp/${sniff}/12000
88     sleep 5
89
90     echo -n "09" >/dev/udp/${sniff}/12000
91     sleep 5
92     echo "> nmap TCP Scan with Random Data (dst_ip:
93 ${IP})"
94
95     sudo nmap --privileged -sT -Pn -n --data-length 24
96 ${IP}
97     echo -n "STOP" >/dev/udp/${sniff}/12000
98     sleep 5
99
100    echo -n "10" >/dev/udp/${sniff}/12000
101    sleep 5
102    echo "> sudo unicornscan TCP SYN Scan (dst_ip:
103 ${IP})"
104
105    sudo unicornscan -Iv -mT -R $repeat_unicornscan
106 ${IP} -i $attacker_interface
107    echo -n "STOP" >/dev/udp/${sniff}/12000
108    sleep 5
109
110    echo -n "11" >/dev/udp/${sniff}/12000
111    sleep 5
112    echo "> sudo unicornscan TCP Connect Scan (dst_ip:
113 ${IP})"
114
115    sudo unicornscan -Iv -msf -R $repeat_unicornscan
116 ${IP} -i $attacker_interface
117    echo -n "STOP" >/dev/udp/${sniff}/12000
118    sleep 5

```

```

119
120     echo -n "12" >/dev/udp/${sniff}/12000
121     sleep 5
122     echo "> sudo unicornscan TCP NULL Scan (dst_ip:
123 ${IP})"
124
125     sudo unicornscan -Iv -mTs -R $repeat_unicornscan
126 ${IP} -i $attacker_interface
127     echo -n "STOP" >/dev/udp/${sniff}/12000
128     sleep 5
129
130     echo -n "13" >/dev/udp/${sniff}/12000
131     sleep 5
132     echo "> sudo unicornscan TCP XMAS Scan (dst_ip:
133 ${IP})"
134
135     sudo unicornscan -Iv -mTsFPU -R $repeat_unicornscan
136 ${IP} -i $attacker_interface
137     echo -n "STOP" >/dev/udp/${sniff}/12000
138     sleep 5
139
140     echo -n "14" >/dev/udp/${sniff}/12000
141     sleep 5
142     echo "> sudo unicornscan TCP FIN Scan (dst_ip:
143 ${IP})"
144
145     sudo unicornscan -Iv -mTsF -R $repeat_unicornscan
146 ${IP} -i $attacker_interface
147     echo -n "STOP" >/dev/udp/${sniff}/12000
148     sleep 5
149
150     echo -n "15" >/dev/udp/${sniff}/12000
151     sleep 5
152     echo "> sudo unicornscan TCP ACK Scan (dst_ip:
153 ${IP})"
154
155     sudo unicornscan -Iv -mTsA -R $repeat_unicornscan
156 ${IP} -i $attacker_interface
157     echo -n "STOP" >/dev/udp/${sniff}/12000
158     sleep 5
159
160     echo -n "16" >/dev/udp/${sniff}/12000
161     sleep 5
162     echo "> sudo hping3 TCP SYN Scan (dst_ip: ${IP})"
163
164     sudo hping3 ${IP} --fast -c $hping_pkt_count -V -p

```

```

165 ++1 -S
166     echo -n "STOP" >/dev/udp/${sniff}/12000
167     sleep 5
168
169     echo -n "17" >/dev/udp/${sniff}/12000
170     sleep 5
171     echo "> sudo hping3 TCP NULL Scan (dst_ip: ${IP})"
172
173     sudo hping3 ${IP} --fast -c $hping_pckt_count -V -p
174 ++1 -Y
175     echo -n "STOP" >/dev/udp/${sniff}/12000
176     sleep 5
177
178     echo -n "18" >/dev/udp/${sniff}/12000
179     sleep 5
180     echo "> sudo hping3 TCP XMAS Scan (dst_ip: ${IP})"
181
182     sudo hping3 ${IP} --fast -c $hping_pckt_count -V -p
183 ++1 -UPF
184     echo -n "STOP" >/dev/udp/${sniff}/12000
185     sleep 5
186
187     echo -n "19" >/dev/udp/${sniff}/12000
188     sleep 5
189     echo "> sudo hping3 TCP FIN Scan (dst_ip: ${IP})"
190
191     sudo hping3 ${IP} --fast -c $hping_pckt_count -V -p
192 ++1 -F
193     echo -n "STOP" >/dev/udp/${sniff}/12000
194     sleep 5
195
196     echo -n "20" >/dev/udp/${sniff}/12000
197     sleep 5
198     echo "> sudo hping3 TCP ACK Scan (dst_ip: ${IP})"
199
200     sudo hping3 ${IP} --fast -c $hping_pckt_count -V -p
201 ++1 -A
202     echo -n "STOP" >/dev/udp/${sniff}/12000
203     sleep 5
204
205     echo -n "21" >/dev/udp/${sniff}/12000
206     sleep 5
207     echo "> sudo masscan scan to network: ${IP_masscan}"
208     sudo masscan --ports 0-1000 $IP_masscan -i
209     $attacker_interface
210     echo -n "STOP" >/dev/udp/${sniff}/12000

```

211	sleep 5
212	done < "\$target"
213	
214	echo "> Finishing scan"

Tahap pengumpulan *traffic reconnaissance* menghasilkan total 21 berkas pcap, sesuai dengan variasi pemindaian yang dilakukan. Namun, hanya 9 dari 21 teknik pemindaian yang digunakan sebagai data latih sesuai Tabel 3.2. Seluruh berkas hasil teknik pemindaian pada Tabel 3.2 diproses melalui tahapan ekstraksi fitur menggunakan kode pada Tabel 4.4.

Tabel 4.4 Pseudocode ekstraksi fitur *traffic reconnaissance*

Extract Reconnaissance Traffic	
1	DEFINE labels AS DICTIONARY {
2	"01": "nmap_syn", "02": "nmap_connect",
3	"03": "nmap_null", "04": "nmap_xmas",
4	"05": "nmap_fin", "06": "nmap_ack",
5	"07": "nmap_window", "08": "nmap_maimon",
6	"09": "nmap_connect_data_length_24",
7	"10": "unicorn_syn", "11": "unicorn_conn",
8	"12": "unicorn_null", "13": "unicorn_xmas",
9	"14": "unicorn_fin", "15": "unicorn_ack",
10	"16": "hping_syn", "17": "hping_null",
11	"18": "hping_xmas", "19": "hping_fin",
12	"20": "hping_ack", "21": "masscan"
13	}
14	
15	DEFINE attributes AS LIST [
16	"frame.encap_type", "frame.time",
17	"frame.time_epoch", "frame.number",
18	"frame.len", "frame.cap_len", "eth.type",
19	"ip.version", "ip.hdr_len", "ip.tos", "ip.id",
20	"ip.flags", "ip.flags.rb", "ip.flags.df",
21	"ip.flags.mf", "ip.frag_offset", "ip.ttl",
22	"ip.proto", "ip.checksum", "ip.src", "ip.dst",
23	"ip.len", "ip.dsfield",
24	"tcp.srcport", "tcp.dstport", "tcp.seq", "tcp.ack",
25	"tcp.len", "tcp.hdr_len", "tcp.flags",
26	"tcp.flags.fin", "tcp.flags.syn", "tcp.flags.reset",
27	"tcp.flags.push", "tcp.flags.ack", "tcp.flags.urg",
28	"tcp.flags.cwr", "tcp.window_size", "tcp.checksum",
29	"tcp.urgent_pointer", "tcp.options.mss_val",
30	"tcp.options.wscale", "tcp.options.sackok"
31	]
32	

```

33 FUNCTION retrieve_attributes(packet, file_num)
34     INITIALIZE pkt_data AS DICTIONARY
35
36     FOR EACH attribute IN attributes
37         SET pkt_data[attribute] TO RETRIEVE attribute
38 FROM packet
39     END FOR
40     SET pkt_data["label"] TO labels[file_num]
41
42     RETURN pkt_data
43 END FUNCTION
44
45 FUNCTION process_pcap(pcap_file, file_num)
46     SET packets TO READ PACKET FROM pcap_file
47     INITIALIZE pcap_ext AS LIST
48
49     FOR EACH packet IN packets
50         SET packet_ext TO retrieve_attributes(packet,
51 file_num)
52         APPEND packet_ext TO pcap_ext
53     END FOR
54
55     RETURN pcap_ext
56 END FUNCTION
57
58 FUNCTION main()
59     INITIALIZE all_rows AS LIST
60
61     FOR EACH file IN DIRECTORY
62         IF file ENDS WITH ".pcap"
63
64             SET file_num AS FIRST 2 CHAR OF file
65             IF file_num NOT IN labels
66                 PRINT "> Skipping " + pcap_file_path + " -
67 unknown label" + file_num
68                 CONTINUE TO NEXT file
69             END IF
70
71             PRINT "Processing: " + file + "
72 (Label: " + labels[file_num] + ")"
73
74             SET rows TO process_pcap(file, file_num)
75
76
77             APPEND rows TO all_rows
78         END IF

```

79	END FOR
80	
81	IF all_rows IS NOT EMPTY
82	WRITE all_rows TO CSV
83	PRINT "Total packets processed: " + NUMBER OF
84	all_rows
85	ELSE
86	PRINT "No packets found!"
87	END IF
88	END FUNCTION
89	
90	CALL main()

4.2.2 Traffic Normal

Pengumpulan data *traffic* normal diawali dengan mengunduh *dataset* dari situs web MAWILab. Sampel data pada tanggal 19 Desember 2024 dipilih karena merupakan data yang paling baru. Spesifikasi lengkap mengenai data yang didapatkan telah dirincikan pada Tabel 3.3. Tahap selanjutnya adalah penyusunan *syntax* Tshark yang berfungsi untuk melakukan filtrasi *traffic*. *Syntax* ini dibuat dengan mengonversi berkas klasifikasi dengan format CSV menggunakan kode program pada Tabel 4.5.

Tabel 4.5 Pseudocode extract filter

Extract Filter	
1	INITIALIZE csv_path
2	READ CSV FILE at csv_path INTO df
3	SET anomalous_rows FROM df WHERE df["nbDetectors"] IS
4	"anomalous" OR df["nbDetectors"] IS "suspicious"
5	REPLACE MISSING VALUE IN anomalous_rows WITH 0
6	INITIALIZE filter_rule AS "not ("
7	
8	FOR EACH row IN anomalous_rows
9	SET srcIP FROM row["srcIP"]
10	SET srcPort FROM row["srcPort"]
11	SET dstIP FROM row["dstIP"]
12	SET dstPort FROM row["dstPort"]
13	
14	INITIALIZE clause AS " ("
15	IF srcIP != 0
16	APPEND " && ip.src == " + srcIP TO clause
17	END IF
18	
19	IF srcPort != 0


```

20     APPEND " && tcp.srcport == " + srcPort TO clause
21 END IF
22
23 IF dstIP != 0
24     APPEND " && ip.dst == " + dstIP TO clause
25 END IF
26
27 IF dstPort != 0
28     APPEND " && tcp.dstport == " + dstPort TO clause
29 END IF
30
31 APPEND ") || " TO clause
32 APPEND clause TO filter_rule
33 END FOR
34
35 APPEND ")" TO filter_rule
36
37 REPLACE FIRST "&&" AT filter_rule
38 REPLACE LAST "||" AT filter_rule
39
40 WRITE filter_rule TO FILE

```

Kode pada Tabel 4.5 mengawali proses dengan membaca berkas CSV, kemudian mengubahnya menjadi format *dataframe*. Data kemudian disaring sehingga hanya memuat data dengan nilai *anomalous* atau *suspicious* pada kolom *nbDetectors*. Data seperti IP dan *port* sumber maupun tujuan diambil dari setiap baris. Data disusun dan disimpan pada sebuah berkas dengan format *syntax* khusus seperti pada Tabel 4.6.

Tabel 4.6 Potongan *syntax* filtrasi Tshark

Syntax Filter	
1	not ((ip.src == 133.74.109.204 && tcp.srcport == 3964
2	&& ip.dst == 202.15.89.45 && tcp.dstport == 443)
3	(ip.src == 104.16.178.215 && tcp.srcport == 443 &&
4	ip.dst == 150.142.34.235 && tcp.dstport == 53279)
5	(ip.src == 60.127.82.109 && tcp.srcport == 46611 &&
6	ip.dst == 133.11.51.159 && tcp.dstport == 9993))

Berkas *pcap* yang didapatkan dari MAWILab perlu dipecah menjadi segmen yang lebih kecil. Setiap segmen tersebut memuat 1 juta paket. Proses segmentasi ini memanfaatkan bantuan Tshark sesuai *script* pada Tabel 4.7.

Tabel 4.7 Script split pcap

Split Pcap	
1	#!/bin/bash
2	editcap -c \$2 \$1 output.pcap

Script pada Tabel 4.7 membutuhkan dua argumen, yaitu batas jumlah paket dan nama berkas pcap target. Berkas pcap tersebut dipecah menjadi segmen yang lebih kecil sesuai batas jumlah paket yang ditentukan, yakni 1 juta paket. Setiap segmen disimpan menggunakan format penamaan seperti output_1.pcap.

Setiap segmen pcap dilakukan penyaringan setelah proses pemecahan selesai. Setiap segmen hasil pemecahan dilakukan filtrasi menggunakan *syntax* yang telah disiapkan. Mekanisme penyaringan tersebut dilakukan dengan bantuan *script* pada Tabel 4.8.

Tabel 4.8 Script filter pcap

Filter Pcap	
1	#!/bin/bash
2	RULES="`cat filter_rule.txt`"
3	
4	for i in output_*.pcap; do
5	tshark -r \${i%.*}.pcap -w \${i%.*}filtered.pcap -Y
6	"\$RULES"
7	done

Script pada Tabel 4.8 melakukan iterasi terhadap setiap segmen berkas pcap. Tshark membaca dan menyaring paket yang ada dalam segmen pcap pada setiap iterasi. Hasil filtrasi menyisakan *traffic* normal yang telah dibersihkan dari indikasi paket berbahaya. *Traffic* bersih tersebut disimpan dengan format nama output_1_filtered.pcap. Langkah selanjutnya adalah melanjutkan filtrasi TCP dan sampling untuk mendapatkan data latih dan data uji. Langkah ini dilakukan menggunakan *script* pada Tabel 4.9.

Tabel 4.9 Script untuk filter protokol TCP dan merge

Filter TCP and Merge	
1	#!/bin/bash
2	set -euo pipefail
3	
4	mkdir -p data
5	
6	for inpcap in output_*filtered.pcap; do

7	[[-f "\$inpcap"]] continue
8	outpcap="\${inpcap%.pcap}_tcp.pcap"
9	echo "[*] TCP-filtering: \$inpcap -> \$outpcap"
10	
11	tshark -r "\$inpcap" -Y tcp -w "\$outpcap"
12	done
13	echo "[✓] Done. Generated *_tcp.pcap for all matching
14	inputs."
15	
16	echo "[*] Merging all TCP-filtered files..."
17	ls output_*_tcp.pcap > tcp_files_to_merge.txt
18	mergcap -w data/bonafide_tcp.pcap \$(cat
19	tcp_files_to_merge.txt)
20	
21	rm tcp_files_to_merge.txt
22	echo "[✓] Final merged file -> data/bonafide_tcp.pcap"
23	
24	echo "[*] Splitting data/bonafide_tcp.pcap into 100,000
25	packet chunks..."
26	
27	editcap -c 100000 data/bonafide_tcp.pcap
28	data/bonafide_tcp_100k.pcap
29	
30	echo "[✓] Done. Generated data/bonafide_tcp_100k.pcap"

Eksekusi *script* pada Tabel 4.9 menghasilkan segmen berkas pcap berisi 100 ribu paket TCP. Segmen pertama dipilih sebagai data latih dan segmen terakhir sebagai data uji. Segmen pcap pertama perlu dilakukan ekstraksi fitur dan disimpan menjadi format CSV serta dibatasi hingga 55 ribu data. Sementara itu, segmen terakhir yang digunakan sebagai data uji dibiarkan dalam format pcap tetapi dengan dibatasi sekitar 55 ribu paket. Proses ekstraksi fitur yang dilakukan menggunakan kode pada Tabel 4.10.

Tabel 4.10 Pseudocode ekstraksi fitur *traffic* normal

Extract Normal Traffic	
1	DEFINE attributes AS LIST [
2	"frame.encap_type", "frame.time",
3	"frame.time_epoch", "frame.number",
4	"frame.len", "frame.cap_len", "eth.type",
5	"ip.version", "ip.hdr_len", "ip.tos", "ip.id",
6	"ip.flags", "ip.flags.rb", "ip.flags.df",
7	"ip.flags.mf", "ip.frag_offset", "ip.ttl",

```

8      "ip.proto", "ip.checksum", "ip.src", "ip.dst",
9      "ip.len", "ip.dsfield",
10     "tcp.srcport", "tcp.dstport", "tcp.seq", "tcp.ack",
11     "tcp.len", "tcp.hdr_len", "tcp.flags",
12     "tcp.flags.fin", "tcp.flags.syn", "tcp.flags.reset",
13     "tcp.flags.push", "tcp.flags.ack", "tcp.flags.urg",
14     "tcp.flags.cwr", "tcp.window_size", "tcp.checksum",
15     "tcp.urgent_pointer", "tcp.options.mss_val",
16     "tcp.options.wscale", "tcp.options.sackok"
17 ]
18
19 FUNCTION retrieve_attributes(packet)
20     INITIALIZE pkt_data AS DICTIONARY
21
22     FOR EACH attribute IN attributes
23         SET pkt_data[attribute] TO RETRIEVE attribute
24 FROM packet
25     END FOR
26
27     RETURN pkt_data
28 END FUNCTION
29
30 FUNCTION main()
31     SET packets TO READ PACKET FROM PCAP FILE
32     PRINT "Processing " + COUNT OF packets + "
33 packets..."
34
35     INITIALIZE all_rows AS LIST
36     FOR EACH packet in packets
37         SET row TO retrieve_attributes(packet)
38         APPEND row TO all_rows
39
40         IF COUNT OF all_row % 10000 IS 0
41             PRINT "Processed " + COUNT OF all_rows + "
42 packets"
43         END IF
44     END FOR
45
46     IF all_rows IS NOT EMPTY
47         WRITE all_rows TO CSV
48         PRINT "Total packets processed: " + NUMBER OF
49 all_rows
50     ELSE
51         PRINT "No packets found!"
52     END IF
53 END FUNCTION

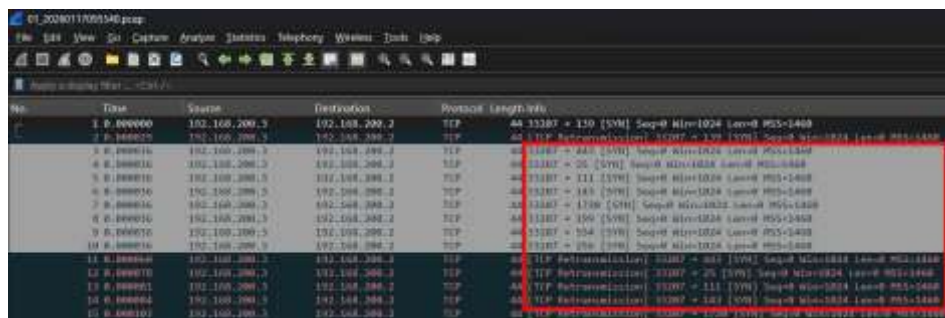
```

54	
55	CALL main()

Tahap pengumpulan data latih dan data uji untuk *traffic* normal telah selesai. *Dataset* latih dengan format CSV sebanyak 55 ribu data digunakan untuk melatih model Decision Tree. Sebaliknya, data uji dengan format pcap berisi sekitar 55 ribu paket akan dikirim ke mesin target. Data tersebut berfungsi sebagai basis evaluasi kinerja sistem pada skenario pengujian. Tahap selanjutnya berfokus pada pengumpulan data terkait aktivitas *reconnaissance*.

4.3 Implementasi Ruleset Suricata

Analisis fitur TCP dilakukan terhadap berkas pcap *traffic reconnaissance*. Observasi difokuskan pada fitur seperti *flag*, *window*, *window scale*, dan *maximum segment size* di setiap paket data. Proses analisis ini memanfaatkan aplikasi Wireshark yang didukung oleh antarmuka grafis. Tampilan antarmuka Wireshark saat memproses salah satu sampel berkas pcap ditampilkan pada Gambar 4.5.



Gambar 4.5 Analisis *traffic* pemindaian menggunakan Wireshark

Analisis dilakukan terhadap berbagai teknik pemindaian, kemudian karakteristik spesifik pada fitur protokol TCP disimpan. Tabel 4.11 menyajikan ringkasan fitur TCP dari setiap teknik pemindaian yang digunakan dalam data latih. Data ini memetakan pola unik pada *header* paket yang membedakan satu teknik pemindaian dengan teknik lainnya. Identifikasi pola unik pada tabel ini berfungsi sebagai landasan utama dalam perancangan *ruleset* Suricata. Pemetaan atribut tersebut memungkinkan pembuatan *signature* yang presisi untuk mendeteksi setiap teknik pemindaian secara spesifik.

Tabel 4.11 Fitur TCP pada data latih *reconnaissance*

No	Teknik Pemindaian	Fitur TCP			
		Flag	Window	wscale	MSS
1	Nmap TCP SYN Scan	S	1024	-	1460
2	Nmap TCP Connect Scan	S	64860	7	1380
		A	64896	-	-
		R+A	64896	-	-

3	Nmap TCP NULL Scan	NULL	1024	-	-
4	Nmap TCP FIN Scan	F	1024	-	-
5	Nmap TCP Maimon Scan	F+A	1024	-	-
6	Unicornscaan TCP XMAS Scan	F+P+U	16384	0	1436
7	Unicornscaan TCP ACK Scan	A	16384	0	1436
8	Hping3 TCP NULL Scan	CWR	512	-	-
9	Hping3 TCP FIN Scan	F	512	-	-

Kumpulan fitur TCP pada Tabel 4.11 merepresentasikan *signature* spesifik dari aktivitas *reconnaissance*. Temuan ini selanjutnya diubah menjadi *ruleset* Suricata untuk memblokir *traffic* berbahaya secara otomatis. Perincian *ruleset* yang telah diimplementasikan dituliskan dalam Tabel 4.12.

Tabel 4.12 Ruleset Suricata untuk teknik pemindaian pada data latih

No	Teknik Pemindaian	Ruleset
1	Nmap TCP SYN Scan	drop tcp any any -> \$HOME_NET any (msg:"NMAP TCP SYN Scan Detected"; flow:stateless,no_stream; tcp.flags:S; window:1024; tcp.mss:1460; classtype:network-recon; sid:1000001; rev:2;)
2	Nmap TCP Connect Scan	drop tcp any any -> \$HOME_NET any (msg:"NMAP TCP Connect Scan Detected"; flow:stateless,no_stream; tcp.flags:S; window:64860; tcp.wscale:7; tcp.mss:1380; classtype:network-recon; sid:1000002; rev:5;) drop tcp any any -> \$HOME_NET any (msg:"NMAP TCP Connect Scan Detected"; flow:stateless,no_stream; tcp.flags:A; window:64896; tcp.wscale:7; classtype:network-recon; sid:1000003; rev:3;) drop tcp any any -> \$HOME_NET any (msg:"NMAP TCP Connect Scan Detected"; flow:stateless,no_stream; tcp.flags:RA; window:64896; classtype:network-recon; sid:1000004; rev:3;)
3	Nmap TCP NULL Scan	drop tcp any any -> \$HOME_NET any (msg:"NMAP TCP NULL Scan Detected"; flow:stateless,no_stream; tcp.flags:0; window:1024; classtype:network-recon; sid:1000005; rev:1;)
4	Nmap TCP FIN Scan	drop tcp any any -> \$HOME_NET any (msg:"NMAP TCP FIN Scan Detected"; flow:stateless,no_stream; tcp.flags:F; window:1024; classtype:network-recon; sid:1000007; rev:1;)

5	Nmap TCP Maimon Scan	drop tcp any any -> \$HOME_NET any (msg:"NMAP Maimon Scan Detected"; flow:stateless,no_stream; tcp.flags:FA; window:1024; classtype:network-recon; sid:1000009; rev:2;)
6	Unicornscaan TCP XMAS Scan	drop tcp any any -> \$HOME_NET any (msg:"Unicornscaan TCP XMAS Scan Detected"; flow:stateless,no_stream; tcp.flags:FPU; window:16384; tcp.wscale:0; tcp.mss:1436; classtype:network-recon; sid:1000012; rev:3;)
7	Unicornscaan TCP ACK Scan	drop tcp any any -> \$HOME_NET any (msg:"Unicornscaan TCP ACK Scan Detected"; flow:stateless,no_stream; tcp.flags:A; window:16384; tcp.wscale:0; tcp.mss:1436; classtype:network-recon; sid:1000014; rev:3;)
8	Hping3 TCP NULL Scan	drop tcp any any -> \$HOME_NET any (msg:"Hping3 TCP NULL Scan Detected"; flow:stateless,no_stream; tcp.flags:C; window:512; classtype:network-recon; sid:1000016; rev:2;)
9	Hping3 TCP FIN Scan	drop tcp any any -> \$HOME_NET any (msg:"Hping3 TCP FIN Scan Detected"; flow:stateless,no_stream; tcp.flags:F; window:512; classtype:network-recon; sid:1000018; rev:2;)

Sebanyak 12 *ruleset* dikembangkan untuk memblokir 9 teknik pemindaian. Jumlah *ruleset* yang melebihi teknik pemindaian dikarenakan Nmap TCP Connect memiliki tiga variasi *signature* unik. Mekanisme deteksi bekerja dengan mencocokkan paket TCP yang mengarah ke IP pada variabel `HOME_NET`. *Syntax* `flow:stateless,no_stream` digunakan untuk memastikan bahwa *signature* dianalisis berbasis paket, bukan *flow*. Jika suatu paket terdeteksi sesuai dengan *ruleset* maka langsung diblokir. Tahap selanjutnya adalah penyesuaian konfigurasi untuk menjalankan Suricata. Konfigurasi Suricata yang diterapkan dalam penelitian ini disajikan pada Tabel 4.13.

Tabel 4.13 Config Suricata yang digunakan

Suricata Config	
1	%YAML 1.1
2	---
3	vars:
4	address-groups:
5	HOME_NET: "[192.168.200.0/24]"
6	EXTERNAL_NET: "any"
7	
8	default-rule-path: "/etc/suricata/rules_skripsi"
9	rule-files:
10	- network_recon.rules
11	

12	exception-policy: ignore
13	
14	nfq:
15	mode: route
16	route-queue: 1
17	fail-open: no
18	
19	stream:
20	inline: yes
21	midstream: true
22	midstream-policy: ignore
23	drop-invalid: no
24	
25	logging:
26	default-log-level: info
27	
28	outputs:
29	- eve-log:
30	enabled: yes
31	filetype: regular
32	filename: "/etc/suricata/logs/recon/eve.json"
33	community-id: true
34	community-id-seed: 0
35	types: [stats, alert]
36	- fast:
37	enabled: yes
38	filename: "/etc/suricata/logs/recon/fast.log"

Konfigurasi Suricata pada Tabel 4.13 mendefinisikan variabel jaringan (seperti `HOME_NET`) untuk mengelompokkan IP. Pendekatan ini memungkinkan penulisan *ruleset* yang lebih dinamis berbasis variabel. Selain itu, konfigurasi ini juga menetapkan lokasi penyimpanan *ruleset* yang digunakan. *Syntax* `mode:route` memerintahkan Suricata untuk meneruskan paket yang tidak diblokir. Paket-paket ini diteruskan menuju Nfqueue 1 sesuai `route-queue`. Aspek krusial lainnya adalah pengaturan mekanisme *logging* menggunakan *fast log* dan *EVE log*.

Implementasi Suricata pada penelitian ini difokuskan pada mekanisme pemeriksaan berbasis paket tunggal. Konsekuensinya, penerapan *ruleset* dan *policy* yang melibatkan *flow* ditiadakan. Parameter konfigurasi seperti `exception-policy`, `midstream-policy`, dan `drop-invalid` diatur dalam status nonaktif. Penyesuaian teknis ini mencegah pemblokiran otomatis terhadap paket yang tidak memiliki status *flow* yang jelas.

4.4 Implementasi Model Decision Tree

Implementasi model Decision dibagi dalam 3 tahap yakni pemrosesan data, *hyperparameter tuning*, kemudian pelatihan dan *deployment* model.

4.4.1 Pemrosesan Data

Tahapan pemrosesan data dimulai dengan penggabungan 2 berkas CSV yang diperoleh dari tahap pengumpulan data. Kolom label dengan nilai *benign* ditambahkan pada data pada *traffic* normal sebelum penggabungan. Nilai *NULL* yang ditemukan dalam *dataset* gabungan kemudian diubah menjadi angka 0. Proses dilanjutkan dengan menghapus data selain protokol TCP dengan menggunakan nilai pada kolom *ip.proto*. Fitur-fitur yang tidak relevan juga dihapus dari data. Implementasi pemrosesan data diimplementasikan pada Tabel 4.14.

Tabel 4.14 Pseudocode preprocessing bagian 1

Preprocessing 1	
1	SET malicious_df TO READ CSV FILE
2	SET bonafide_df TO READ CSV FILE
3	
4	SET bonafide_df["label"] TO "benign"
5	
6	SET full_data TO CONCATENATE bonafide_df AND
7	malicious_df
8	
9	SET full_data TO REPLACE NaN WITH 0 IN full_data
10	
11	SET wrong_proto_counts TO COUNT VALUES OF
12	full_data WHERE full_data["ip.proto"] != 6
13	
14	FILTER full_data WHERE full_data["ip.proto"] == 6
15	PRINT "Found and removed" + wrong_proto_counts +
16	"packets from the original dataset."
17	
18	DROP COLUMNS FROM full_data:
19	"frame.time", "frame.encap_type",
20	"frame.time_epoch", "frame.number",
21	"frame.len", "frame.cap_len", "eth.type",
22	"ip.flags", "ip.src", "ip.dst",
23	"ip.version", "ip.proto", "tcp.flags"
24	
25	DROP COLUMNS FROM full_data:
26	"ip.ttl", "tcp.seq", "ip.checksum", "tcp.checksum",
27	"tcp.ack", "tcp.dstport",
28	"tcp.srcport", "ip.dsfield", "tcp.urgent_pointer"
29	

Tahap selanjutnya adalah pengecekan variasi nilai pada fitur yang tersisa. Fitur dengan nilai konstan atau tanpa variasi dihapus dari *dataset*. Data dengan nilai duplikat kemudian dihapus. Kolom label dikonversi menjadi format biner nol dan satu sebelum melanjutkan ke proses berikutnya. Implementasi pengecekan variasi nilai tersebut dituliskan pada Tabel 4.15.

Tabel 4.15 Pseudocode preprocessing bagian 2

Preprocessing 2	
1	COMPUTE variance_per_feature FOR ALL COLUMNS EXCEPT
2	"label" IN full_data
3	PRINT variance_per_feature
4	
5	DROP COLUMNS FROM full_data:
6	'ip.hdr_len', 'ip.flags.rb',
7	'ip.flags.mf', 'ip.frag_offset'
8	
9	REMOVE DUPLICATE RECORDS FROM full_data
10	
11	FOR EACH row IN full_data
12	IF row["label"] EQUALS "benign"
13	SET row["label"] TO 0
14	ELSE
15	SET row["label"] TO 1
16	END IF
17	END FOR
18	
19	PRINT full_data INFO

Tahap akhir pemrosesan menerapkan strategi khusus dalam penanganan duplikasi data dan fitur IP ID. Eliminasi data duplikat dieksekusi terlebih dahulu dengan tetap mempertahankan keberadaan fitur IP ID. Kemudian *dataset* disimpan ke format CSV untuk keperluan pelatihan dan *deployment* model. Kode program pada Tabel 4.16 mengimplementasikan prosedur tersebut hingga tahap penyimpanan data.

Tabel 4.16 Pseudocode preprocessing bagian 3

Preprocessing 3	
1	REMOVE DUPLICATE RECORDS FROM full_data
2	
3	DROP COLUMN "ip.id" FROM full_data
4	
5	PRINT full_data INFO

```

6
7  FOR EACH row IN full_data
8      IF row["label"] EQUALS "benign"
9          SET row["label"] TO 0
10     ELSE
11         SET row["label"] TO 1
12     END IF
13 END FOR
14
15 WRITE full_data TO CSV FILE

```

Dataset dalam berkas CSV berhasil dibuat. Fitur-fitur yang terdapat dalam *dataset* ini disajikan dalam Tabel 4.17. Setelah *dataset* siap, tahapan penelitian berlanjut pada optimalisasi model. Optimalisasi pada model Decision Tree dilakukan melalui penerapan teknik *hyperparameter tuning*.

Tabel 4.17 Fitur dalam *dataset* latih model

No	Nama Fitur
1	ip.tos
2	ip.flags.df
3	ip.len
4	tcp.len
5	tcp.hdr_len
6	tcp.flags.fin
7	tcp.flags.syn
8	tcp.flags.reset
9	tcp.flags.push
10	tcp.flags.ack
11	tcp.flags.urg
12	tcp.flags.cwr
13	tcp.window_size
14	tcp.options.mss_val
15	tcp.options.wscale
16	tcp.options.sackok

4.4.2 Hyperparameter Tuning

Hyperparameter tuning pada model Decision Tree menguji seluruh kombinasi *hyperparameter* yang memungkinkan. *Pseudocode* pada Tabel 4.18 mengimplementasikan *nested cross validation* untuk mencari *hyperparameter* optimal. Proses diawali dengan pemisahan atribut fitur dan label target dari *dataset*. Seluruh *dataset* dilakukan *scaling* sebelum *tuning* dilakukan. Setiap kombinasi *hyperparameter* diuji pada *inner cross validation* sebanyak 5 kali iterasi. Kombinasi *hyperparameter* dengan *f1-score* tertinggi

disimpan pada setiap iterasinya. Prosedur ini diulangi dalam 10 iterasi pada *outer cross validation* guna menghasilkan 10 konfigurasi *hyperparameter* terbaik. Keluaran akhir berupa setiap daftar konfigurasi optimal dan skor *f1-score* pada konfigurasi tersebut.

Tabel 4.18 Pseudocode hyperparameter tuning

Hyperparameter Tuning	
1	INITIALIZE dt_params AS DICTIONARY
2	SET dt_params["splitting_criterion"] TO ["gini",
3	"entropy", "log_loss"]
4	SET dt_params["maximum_tree_depth"] TO [5, 10, 15, 20,
5	25, 30, None]
6	SET dt_params["class_weight_option"] TO ["balanced"]
7	SET dt_params["split_strategy"] TO ["best", "random"]
8	
9	SET y TO dataset["label"]
10	SET X TO dataset WITHOUT COLUMN "label"
11	
12	DEFINE outer_cv AS STRATIFIED K-FOLD CROSS
13	VALIDATION WITH NUMBER OF FOLDS TO 10
14	
15	DEFINE inner_cv AS STRATIFIED K-FOLD CROSS
16	VALIDATION WITH NUMBER OF FOLDS TO 5
17	
18	INITIALIZE dt_scores AS LIST
19	INITIALIZE dt_best_params AS LIST
20	
21	FOR EACH train AND test PRODUCED BY outer_cv USING
22	X AND y
23	DEFINE scaler AS STANDART SCALER
24	SET X_train_scaled AS X[train] SCALED BY scaler
25	SET X_test_scaled AS X[test] SCALED BY scaler
26	
27	DEFINE hyperparameter_procedure AS HYPERPARAMETER
28	PROCEDURE USING dt_params AND inner_cv
29	RUN hyperparameter_procedure USING X_train_scaled
30	AND y[train]
31	
32	APPEND BEST PARAM FROM hyperparameter_procedure
33	INTO dt_best_params
34	APPEND F1 SCORE FROM hyperparameter_procedure
35	INTO dt_scores
36	END FOR
37	
38	PRINT dt_best_params
39	PRINT dt_scores

Tabel 4.19 merangkum hasil eksperimen *hyperparameter tuning*. Kombinasi *hyperparameter* pada iterasi pertama dipilih sebagai konfigurasi final karena merupakan varian yang paling sering muncul di antara hasil terbaik. Konfigurasi ini menggunakan Gini *Impurity* sebagai kriteria pemisahan untuk memaksimalkan kemurnian pada setiap simpul pohon. Strategi pemisahan diatur menggunakan metode *Best* guna memastikan pemilihan fitur yang paling optimal di setiap percabangan. Bobot kelas diatur menjadi *Balanced* untuk memprioritaskan kategori minoritas, sehingga model tidak bias terhadap kelas mayoritas. Terakhir, kompleksitas model dikendalikan dengan membatasi kedalaman pohon maksimal hingga 15 level. Batasan ini bertujuan agar model mampu menangkap karakteristik data tanpa mengalami *overfitting* yang signifikan. Kombinasi ini kemudian diterapkan pada tahap pelatihan model final dan proses *deployment*.

Tabel 4.19 Hasil *hyperparameter tuning*

No	Hyperparameter				F1-Score
	Splitting Criterion	Maximum Tree Depth	Class Weight Option	Split Strategy	
1	Gini	15	Balanced	Best	0.9934
2	Entropy	None	Balanced	Random	0.9960
3	Gini	15	Balanced	Best	0.9963
4	Gini	15	Balanced	Best	0.9945
5	Entropy	10	Balanced	Best	0.9945
6	Gini	25	Balanced	Random	0.9942
7	Entropy	10	Balanced	Best	0.9942
8	Entropy	None	Balanced	Random	0.9941
9	Gini	15	Balanced	Best	0.9941
10	Gini	30	Balanced	Random	0.9931

4.4.3 Pelatihan dan *Deployment* Model

Tahapan ini terbagi menjadi dua fase utama. Fase pertama meliputi pelatihan model menggunakan konfigurasi *hyperparameter* optimal, dilanjutkan dengan penyimpanan model ke dalam format berkas spesifik. Fase kedua berfokus pada eksekusi model yang telah tersimpan untuk mengambil keputusan klasifikasi terhadap paket.

Tabel 4.20 *Pseudocode* pelatihan model Decision Tree

Pelatihan Model	
1	DEFINE model_path AS "final_dt_model.pkl"
2	DEFINE scaler_path AS "final_scaler.pkl"
3	

```

4  DEFINE feature_columns AS LIST [
5      "ip.tos", "ip.flags.df", "ip.len", "tcp.len",
6      "tcp.hdr_len", "tcp.flags.fin", "tcp.flags.syn",
7      "tcp.flags.reset", "tcp.flags.push",
8      "tcp.flags.ack", "tcp.flags.urg", "tcp.flags.cwr",
9      "tcp.window_size", "tcp.options.mss_val",
10     "tcp.options.wscale", "tcp.options.sackok"
11 ]
12
13 FUNCTION train_model(dataset_path)
14     PRINT "[*] Starting model training with dataset: " +
15     dataset_path
16
17     IF dataset_path DOES NOT EXIST
18         PRINT "[!] Error: Dataset file not found at " +
19     dataset_path
20         RETURN
21     END IF
22
23     SET df TO READ CSV FILE AT dataset_path
24     PRINT "[*] Dataset loaded successfully with " +
25     NUMBER OF ROWS df + " rows."
26
27     IF "label" NOT IN df COLUMNS
28         PRINT "[!] Error: 'label' column not found in the
29     dataset."
30         RETURN
31     END IF
32
33     SET y TO df["label"]
34     SET X TO df WITHOUT COLUMN "label"
35
36     IF LIST OF COLUMNS X IS NOT EQUAL TO feature_columns
37         PRINT "[!] Error: Dataset columns do not match
38     the expected feature list."
39         PRINT "    Expected: " + feature_columns
40         PRINT "    Got:      " + LIST OF COLUMNS X
41         EXIT
42     END IF
43
44     PRINT "[*] Scaling features..."
45     DEFINE scaler AS STANDARD SCALER
46     SET X_scaled TO X SCALED BY scaler
47
48     PRINT "[*] Training the Decision Tree classifier on
49     the full dataset..."

```

```

50     DEFINE dt_model AS DECISION TREE MODEL (
51         class_weight = "balanced"
52         criterion     = "gini"
53         max_depth     = 15
54         splitter      = "best"
55         random_state  = 47
56     )
57
58     TRAIN dt_model USING X_scaled AND y
59     PRINT "[*] Model training complete."
60
61     WRITE dt_model TO model_path FILE
62     WRITE scaler TO scaler_path FILE
63
64     PRINT "[*] Model trained on the entire dataset and
65 saved to " + model_path
66     PRINT "[*] Scaler saved to " + scaler_path
67 END FUNCTION
68
69 IF CODE IS RUNNING
70     REQUIRED ARGUMENT "dataset" WITH HELP TEXT "Path
71 to the CSV dataset file."
72     CALL train_model(dataset)
73 END IF

```

Eksekusi kode pada Tabel 4.20 memerlukan *input* nama berkas CSV yang digunakan sebagai *dataset*. Validasi awal dilakukan untuk memeriksa kolom label dan kesesuaian struktur data. Jika validasi gagal, proses eksekusi akan dihentikan secara otomatis untuk mencegah kesalahan. *Dataset* yang valid selanjutnya melalui proses standardisasi fitur sebelum digunakan untuk melatih model. Model dilatih menggunakan konfigurasi *hyperparameter* optimal, lalu disimpan dalam format berkas *serialized*. Obyek standardisasi juga disimpan untuk menjamin konsistensi pengolahan data pada tahap *deployment*.

Pada tahap *deployment*, artefak model dan obyek standardisasi memerlukan mekanisme pendukung untuk beroperasi. Diperlukan fungsi khusus yang bertugas mengekstraksi fitur dari paket jaringan. Data fitur tersebut kemudian dilakukan standardisasi sebelum diproses untuk pengambilan keputusan klasifikasi. Fungsi ini berjalan pada antrean *Nfqueue* 1 agar dapat menganalisis paket yang diteruskan oleh *Suricata*. Implementasi kode untuk mekanisme integrasi dan *deployment* terdapat pada Tabel 4.21.

Tabel 4.21 Pseudocode integrasi dan deployment model

Integration and Deployment Model

```

1  TRY
2      CHECK REQUIRED DEPENDENCIES
3  CATCH ImportError
4      PRINT "Dependencies not found. Please install
5  required packages."
6      EXIT
7  END TRY
8
9  DEFINE model_path AS "final_dt_model.pkl"
10 DEFINE scaler_path AS "final_scaler.pkl"
11
12 INITIALIZE model
13 INITIALIZE scaler
14
15 DEFINE feature_columns AS LIST [
16     "ip.tos", "ip.flags.df", "ip.len", "tcp.len",
17     "tcp.hdr_len", "tcp.flags.fin", "tcp.flags.syn",
18     "tcp.flags.reset", "tcp.flags.push",
19     "tcp.flags.ack", "tcp.flags.urg", "tcp.flags.cwr",
20     "tcp.window_size", "tcp.options.mss_val",
21     "tcp.options.wscale", "tcp.options.sackok"
22 ]
23
24 FUNCTION process_packet(packet)
25     TRY
26         IF packet IS TCP PROTOCOL
27             INITIALIZE pkt_data AS DICTIONARY
28             FOR EACH attribute IN feature_columns
29                 SET pkt_data[attribute] TO RETRIEVE
30 attribute FROM packet
31             END FOR
32
33             SET scaled_features TO SCALED pkt_data
34 USING scaler
35             SET prediction TO PREDICT scaled_features
36 USING model
37             IF prediction == 1
38                 CALL log_malicious_packet(
39                     ip_src = packet IP SOURCE,
40                     port_src = packet PORT SOURCE,
41                     ip_dest = packet IP DESTINATION,
42                     port_dest= packet PORT DESTINATION
43                 )
44                 DROP packet
45             ELSE
46                 ACCEPT packet

```



```

47         END IF
48
49     ELSE
50         ACCEPT packet
51     END IF
52
53     CATCH ANY Exception AS e
54         PRINT "Error processing packet: " + e
55         ACCEPT packet
56     END TRY
57 END FUNCTION
58
59 FUNCTION log_malicious_packet(
60     ip_src, port_src, ip_dest, port_dest,
61     protocol="TCP",
62     action="Drop",
63     message="Malicious Reconnaissance Detected",
64     log_file=" IPS_LOG_FILE"
65 )
66
67     SET date TO CURRENT DATE
68     SET time TO CURRENT TIME
69
70     CONSTRUCT log_entry AS JSON
71
72     WRITE log_entry + NEW LINE TO log_file
73
74 END FUNCTION
75
76 FUNCTION run_ips()
77     PRINT "[*] Starting Layer 2 ML IPS..."
78
79     IF model_path OR scaler_path DOES NOT EXIST
80         PRINT "[!] Error: Model or scaler files not
81 found. Train the model first."
82         EXIT
83     END IF
84
85     TRY
86         SET model TO LOAD model_path
87         SET scaler TO LOAD scaler_path
88         PRINT "[*] Model and scaler loaded successfully."
89     CATCH ANY Exception AS e
90         PRINT "[!] Error loading model or scaler: " + e
91         EXIT
92     END TRY

```

```

93
94     TRY
95         INITIALIZE queue AS NetfilterQueue INSTANCE
96         BIND queue TO NFQUEUE NUMBER 1 WITH
97         CALLBACK process_packet
98
99         PRINT "[*] Listening on NFQUEUE 1... Press Ctrl+C
100 to exit."
101         RUN queue EVENT LOOP
102         CATCH PermissionError
103             PRINT "[!] Permission denied. Please run with
104 administrator privileges."
105         CATCH OSError AS e
106             PRINT "[!] OSError: " + e + ". Verify packet
107 filtering modules are available and loaded."
108         CATCH KeyboardInterrupt
109             PRINT "[*] Shutting down IPS."
110         FINALLY
111             IF queue EXISTS
112                 UNBIND queue
113             END IF
114         END TRY
115     END FUNCTION
116
117 CALL run_ips()

```

Kode pada Tabel 4.20 memulai sistem dengan memuat artefak model Decision Tree dan obyek standardisasi yang telah tersimpan sebelumnya. Sebuah fungsi utama beroperasi pada Nfqueue 1 untuk menghentikan paket jaringan yang lewat. Fitur paket diekstraksi dan dianalisis oleh model untuk menghasilkan keputusan klasifikasi final. Sistem secara otomatis memblokir paket dan mencatat ke dalam berkas *logging* apabila paket teridentifikasi sebagai aktivitas *reconnaissance*.

4.5 Implementasi Skenario Pengujian

Sub-bab ini menguraikan tahapan implementasi untuk seluruh skenario pengujian. Rangkaian pengujian ini meliputi skenario Suricata tunggal, model *Decision Tree* tunggal, serta mekanisme *hybrid* IPS. Pengambilan data berfokus pada kinerja deteksi melalui *confusion matrix* serta pengukuran *throughput* jaringan.

Konfigurasi pada mesin IPS perlu disesuaikan sebelum pelaksanaan setiap skenario pengujian. Penyesuaian tersebut diterapkan secara spesifik mengikuti kebutuhan skenario yang dijalankan. *Script* pada Tabel 4.22 digunakan pada skenario pengujian Suricata tunggal. *Script* ini berfungsi untuk mengarahkan *traffic*

jaringan menuju Suricata, sekaligus menjalankannya menggunakan konfigurasi khusus.

Tabel 4.22 Script skenario Suricata tunggal

Suricata Only IPS	
1	#!/usr/bin/env bash
2	set -euo pipefail
3	TARGET_IP=192.168.200.2/32
4	
5	echo "[*] Menerapkan iptables rules"
6	
7	iptables -t mangle -D PREROUTING -i wg0
8	-d "\$TARGET_IP" -j NFQUEUE --queue-num 0 2>/dev/null
9	true
10	
11	iptables -t mangle -A PREROUTING -i wg0
12	-d "\$TARGET_IP" -j NFQUEUE --queue-num 0
13	
14	systemctl stop suricata true
15	suricata -D -c /etc/suricata/suricata-recon-only.yaml
16	-q 0
17	
18	echo "[*] Suricata RECON berjalan @Q0."
19	echo "[*] Hybrif FW aktif. ML harus bind ke Q1."

Suricata berjalan dengan konfigurasi pada Tabel 4.23 untuk skenario Suricata tunggal. Konfigurasi `mode:accept` membuat Suricata hanya melakukan pemeriksaan *signature* terhadap *ruleset*. Paket data yang tidak memiliki kecocokan *signature* akan diizinkan lewat tanpa pengecekan lanjutan.

Tabel 4.23 Konfigurasi Suricata tunggal

Suricata Only Config	
1	%YAML 1.1
2	---
3	vars:
4	address-groups:
5	HOME_NET: "[192.168.200.0/24]"
6	EXTERNAL_NET: "any"
7	
8	default-rule-path: "/etc/suricata/rules_skripsi"
9	rule-files:
10	- network_recon.rules
11	

12	exception-policy: ignore
13	
14	nfq:
15	mode: accept
16	fail-open: no
17	
18	stream:
19	inline: yes
20	midstream: true
21	midstream-policy: ignore
22	drop-invalid: no
23	
24	logging:
25	default-log-level: info
26	
27	outputs:
28	- eve-log:
29	enabled: yes
30	filetype: regular
31	filename: "/etc/suricata/logs/recon/eve.json"
32	community-id: true
33	community-id-seed: 0
34	types: [stats, alert]
35	- fast:
36	enabled: yes
37	filename: "/etc/suricata/logs/recon/fast.log"

Tahapan pengujian selanjutnya difokuskan pada penerapan skenario model Decision Tree tunggal. Pelaksanaan skenario ini dijalankan menggunakan *script* yang tercantum pada Tabel 4.24. Iptables diterapkan untuk meneruskan paket secara langsung menuju antrian Nfqueue milik model. Proses ekstraksi fitur dan klasifikasi pada komponen Decision Tree tetap menggunakan kode program pada Tabel 4.20.

Tabel 4.24 Script skenario Decision Tree tunggal

Decision Tree Only IPS	
1	#!/usr/bin/env bash
2	set -euo pipefail
3	TARGET_IP=192.168.200.2/32
4	
5	echo "[*] Menerapkan iptables rules"
6	
7	iptables -t mangle -D PREROUTING -i wg0
8	-d "\$TARGET_IP" -j NFQUEUE --queue-num 1 2>/dev/null

9	true
10	
11	iptables -t mangle -A PREROUTING -i wg0
12	-d "\$TARGET_IP" -j NFQUEUE --queue-num 1
13	
14	systemctl stop suricata true
15	
16	echo "[*] Suricata RECON berjalan @Q0."
17	echo "[*] Hybrif FW aktif. ML harus bind ke Q1."

Pengujian *throughput* dijalankan menggunakan aplikasi iperf3 yang dijalankan melalui *script* khusus. Implementasi ini melibatkan dua jenis *script* berbeda yang dieksekusi secara terpisah pada mesin *user* serta dan target. *Script* pada Tabel 4.25 secara otomatis menjalankan iperf3 *client* sekaligus menyimpan seluruh hasil pengujian ke dalam berkas *logging*. *Syntax* pada baris ke-12 berfungsi memvalidasi jumlah iterasi pengujian yang sedang dijalankan. *Script* ini menerapkan jeda waktu selama 60 detik apabila iterasi belum mencapai iterasi kesepuluh.

Tabel 4.25 Script iperf3 pada user

Iperf3 client	
1	#!/usr/bin/env bash
2	set -euo pipefail
3	
4	mkdir -p ./iperf_logs
5	
6	for ((i=1; i<=10; i++)); do
7	echo "Run \${i}/10 starting..."
8	iperf3 -c 192.168.200.2 -p 5201 -t 100 -i 1 tee -a
9	./iperf_logs/iperf3_client.log
10	echo "Run \${i}/10 finished."
11	
12	if ["\$i" -lt 10]; then
13	sleep 60
14	fi
15	done
16	
17	echo "All runs completed."

Eksekusi iperf3 pada mesin target dijalankan menggunakan kode *script* yang tercantum pada Tabel 4.26. *Script* tersebut dirancang untuk menjalankan iperf3 secara kontinu hingga dihentikan secara manual. Iperf3 secara otomatis dimatikan dan dihidupkan ulang segera setelah satu iterasi pengujian *client* selesai. Seluruh

data hasil pengujian yang diperoleh selama proses eksekusi disimpan ke dalam berkas *logging*.

Tabel 4.26 Script iperf3 pada target

Iperf3 server	
1	#!/usr/bin/env bash
2	set -euo pipefail
3	
4	mkdir -p ./iperf_logs
5	
6	echo "iperf3 server starting..." tee -a
7	./iperf_logs/iperf3_server.log
8	
9	while true; do
10	echo "Test starting..." tee -a
11	./iperf_logs/iperf3_server.log
12	iperf3 -s -1 -B 192.168.200.2 -p 5201 tee -a
13	./iperf_logs/iperf3_server.log
14	echo "Test finished." tee -a
15	./iperf_logs/iperf3_server.log
16	done

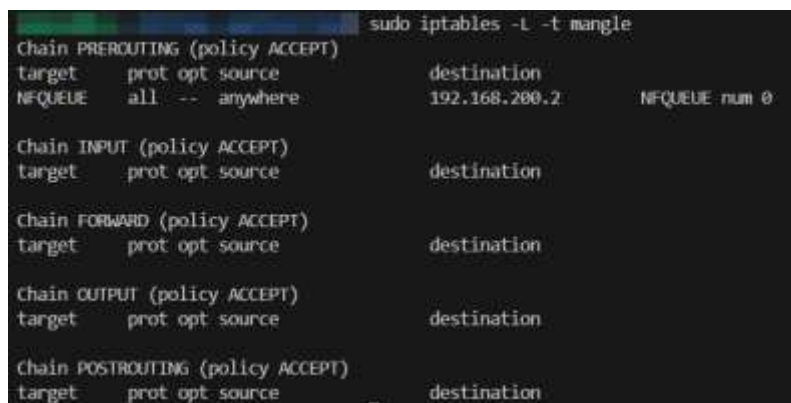
BAB 5 HASIL DAN ANALISIS

Bab ini menjelaskan hasil dan analisis yang didapatkan selama penelitian. Pembahasan difokuskan untuk menjawab rumusan masalah penelitian. Sub bab hasil berisi data-data implementasi *hybrid* IPS dan kinerja yang didapatkan. Sub bab berikutnya berisi analisis dari data yang didapatkan.

5.1 Hasil

5.1.1 Hasil Implementasi *hybrid* IPS

Langkah pertama implementasi *hybrid* IPS adalah meneruskan paket yang datang ke Nfqueue untuk diproses Suricata. Gambar 5.1 menunjukkan aturan *PREROUTING* pada *iptables*. Aturan ini mengarahkan paket dengan tujuan IP target ke antrean Nfqueue 0. Dengan begitu, Suricata yang berjalan pada Nfqueue 0 dapat melakukan pengecekan terhadap *ruleset*.



```
sudo iptables -L -t mangle
Chain PREROUTING (policy ACCEPT)
target prot opt source destination
NFQUEUE all -- anywhere 192.168.200.2 NFQUEUE num 0
Chain INPUT (policy ACCEPT)
target prot opt source destination
Chain FORWARD (policy ACCEPT)
target prot opt source destination
Chain OUTPUT (policy ACCEPT)
target prot opt source destination
Chain POSTROUTING (policy ACCEPT)
target prot opt source destination
```

Gambar 5.1 Aturan iptables pada mesin IPS

Gambar 5.2 menunjukkan proses dari setiap komponen *hybrid* IPS. Suricata berjalan dengan konfigurasi yang telah dibuat dan berjalan pada Nfqueue 0 untuk menerima paket dari iptables. Selain itu, kode program berjalan untuk melakukan ekstraksi fitur dan klasifikasi terhadap paket. Paket yang diblokir pada setiap komponen akan tercatat melalui mekanisme *logging*.



```
root 14489 suricata -D -c /etc/suricata/suricata-recon.yaml -q 0
root 14502 sudo .venv/bin/python3 final_ml_firewall_run.py
root 14503 sudo .venv/bin/python3 final_ml_firewall_run.py
```

Gambar 5.2 Komponen *hybrid* IPS yang berjalan

Hasil *logging* dari komponen Suricata dan Decision Tree terdapat pada Gambar 5.3 dan 5.4. Kedua berkas tersebut menyimpan informasi waktu paket di blokir dan pesan blokir. Selain itu, informasi mengenai paket seperti IP serta *port* sumber dan tujuan juga disimpan.

Hasil *confusion matrix* pada skenario Decision Tree tunggal ditampilkan dalam Tabel 5.2. Total keseluruhan paket yang diproses dalam skenario pengujian ini berjumlah 86679 paket. Data tersebut terdiri dari 29181 paket dengan kategori *reconnaissance* dan 57498 kategori normal. Model tercatat berhasil mendeteksi dan memblokir paket *reconnaissance* sebanyak 21142. Sementara itu, terdapat 8039 paket *reconnaissance* lainnya yang gagal dideteksi oleh sistem. Sebanyak 57267 paket dengan kategori normal berhasil diteruskan menuju target. Sistem hanya melakukan kesalahan pemblokiran terhadap 231 paket yang seharusnya bersifat normal.

Tabel 5.2 *Confusion matrix* skenario Decision Tree tunggal

Aktual/Prediksi	Reconnaissance	Normal
Reconnaissance	21142	8039
Normal	231	57267

Hasil *confusion matrix* untuk skenario *hybrid* IPS dipaparkan pada Tabel 5.3. Jumlah total paket yang terlibat dalam proses pengujian skenario gabungan ini adalah 86704. Data tersebut terbagi menjadi 29205 paket *reconnaissance* dan 57499 paket normal. Sistem *hybrid* IPS berhasil mendeteksi dan memblokir paket *reconnaissance* sebanyak 21172 paket dari total *reconnaissance*. Sementara itu, terdapat sisa 8033 paket *reconnaissance* yang gagal diblokir dan lolos deteksi. Sebanyak 56764 paket normal berhasil diteruskan sistem menuju target. Adapun paket normal yang terblokir secara keliru oleh *hybrid* IPS tercatat sejumlah 735 paket.

Tabel 5.3 *Confusion matrix* skenario *hybrid* IPS

Aktual/Prediksi	Reconnaissance	Normal
Reconnaissance	21172	8033
Normal	735	56764

Hasil *confusion matrix* dikonversi menjadi metrik evaluasi kinerja yang terlampir pada Tabel 5.4. Skenario Suricata tunggal mencatatkan nilai akurasi sebesar 0.8310 serta presisi mencapai 0.9589. Adapun nilai *recall* pada skenario ini adalah 0.5338 dengan *f1-score* 0.6858. Implementasi model *Decision Tree* tunggal memperoleh akurasi 0.9046 dan presisi setinggi 0.9892. Metrik *recall* skenario ini tercatat di angka 0.7245 serta *f1-score* sebesar 0.8364. Pengujian terakhir pada skenario *hybrid* IPS menunjukkan capaian akurasi sebesar 0.8989. Nilai presisi tercatat sebesar 0.9664, diikuti dengan *recall* 0.7249 dan *f1-score* 0.8285.

Tabel 5.4 Hasil metrik evaluasi

Skenario	Akurasi	Presisi	<i>Recall</i>	<i>F1-score</i>
Suricata tunggal	0.8314	0.9576	0.5338	0.6855

Decision Tree tunggal	0.9046	0.9892	0.7245	0.8364
Hybrid IPS	0.8989	0.9664	0.7249	0.8285

5.1.3 Hasil *Throughput* Jaringan

Pengujian *throughput* dilakukan untuk mengukur kapasitas pengiriman data maksimal pada setiap skenario. Skenario *baseline* menghasilkan nilai rata-rata sebesar 861.30 Mbps. Variasi nilai *throughput* pada skenario ini berkisar antara 847 hingga 876 Mbps. Implementasi Suricata tunggal menghasilkan rata-rata *throughput* sebesar 674.70 Mbps selama pengujian berlangsung. Fluktuasi nilai pada skenario ini tercatat bergerak stabil dari 663 hingga 684 Mbps. Skenario Decision Tree tunggal mencatatkan capaian rata-rata yang jauh lebih rendah di angka 9.648 Mbps. Rentang nilai pada skenario ini bergerak stabil antara 9.55 sampai 9.77 Mbps. Pengujian skenario *hybrid IPS* menunjukkan hasil rata-rata *throughput* sebesar 9.497 Mbps. Nilai minimum yang terukur adalah 9.35 Mbps dengan nilai tertinggi mencapai 9.60 Mbps. Data hasil pengujian *throughput* selengkapnya dapat dilihat pada Tabel 5.5.

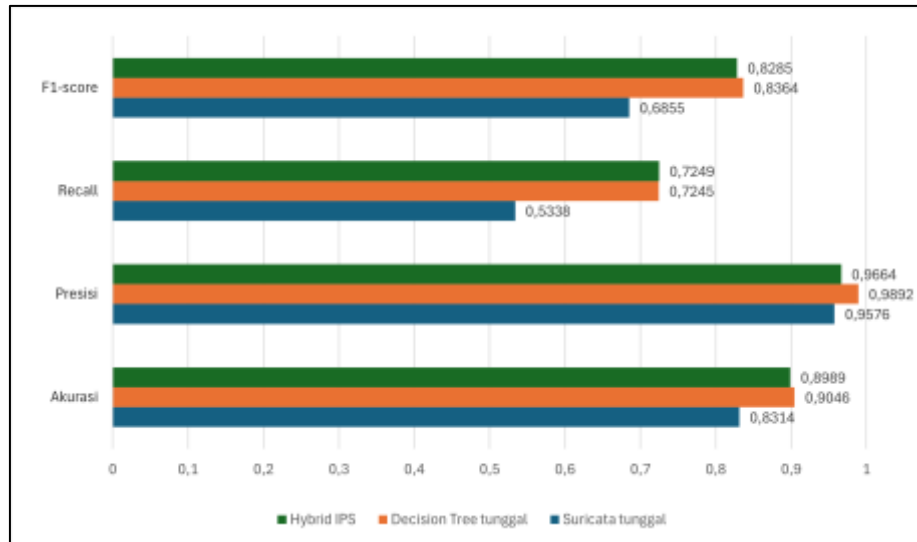
Tabel 5.5 Hasil pengujian *throughput* tiap skenario

Skenario	Rata-Rata	Nilai Terkecil	Nilai Terbesar
Baseline	861.30	847	876
Suricata tunggal	674.70	663	684
Decision Tree tunggal	9.648	9.55	9.77
Hybrid IPS	9.497	9.35	9.60

5.2 Analisis

5.2.1 Analisis Perbandingan Kinerja IPS

Analisis kinerja pada penelitian ini menitikberatkan pada keseimbangan antara kemampuan deteksi dan tingkat kesalahan prediksi. Ketiga skenario pengujian menunjukkan karakteristik *trade-off* yang berbeda antara metrik *recall* dan presisi. Suricata cenderung menunjukkan karakteristik deteksi yang sangat selektif dengan cakupan yang terbatas. Di sisi lain, model Decision Tree menawarkan keseimbangan performa yang jauh lebih stabil. *Hybrid IPS* justru memperlihatkan pendekatan agresif dengan risiko kesalahan deteksi yang sedikit meningkat dibandingkan Decision Tree tunggal. Perbandingan metrik kinerja antar skenario dapat dilihat pada Gambar 5.5.



Gambar 5.5 Perbandingan metrik evaluasi pada setiap skenario

Skenario Suricata tunggal menghasilkan kinerja yang kurang optimal, terutama pada aspek kemampuan deteksi serangan. Hal ini terlihat jelas dari nilai *recall* yang hanya mampu mencapai angka 0.5338. Suricata gagal mendeteksi dan memblokir lebih dari 14000 ribu paket *reconnaissance*. Kondisi ini menandakan kecenderungan Suricata tunggal lebih ketat terhadap *signature* yang benar-benar dikenali. Namun, belum cukup luas cakupannya untuk menjangkau variasi teknik pemindaian yang digunakan pada data uji. Banyaknya serangan yang lolos ini menunjukkan adanya celah signifikan dalam mekanisme pertahanan *signature*.

Rendahnya tingkat deteksi pada Suricata disebabkan oleh ketidakcocokan spesifik antara *signature* teknik *reconnaissance* dengan *ruleset* yang aktif. Suricata bekerja dengan mencocokkan *signature* secara statis, sehingga variasi *reconnaissance* yang sedikit berbeda dapat mengecoh sistem. Paket *reconnaissance* yang digunakan memiliki karakteristik yang tidak terdaftar dalam *ruleset*. Akibatnya, *traffic* berbahaya tersebut dianggap sebagai aktivitas normal dan diizinkan masuk. Tabel 5.6 menunjukkan karakteristik teknik pemindaian yang lolos dari *ruleset* Suricata.

Tabel 5.6 Karakteristik teknik pemindaian tidak berhasil dideteksi oleh Suricata

No	Teknik Pemindaian	Fitur TCP			
		Flag	Window	wscale	MSS
1	Nmap TCP XMAS Scan	F+P+U	1024	-	-
2	Nmap TCP ACK Scan	A	1024	-	-
3	Nmap TCP Window Scan				
4	Unicornscaan TCP SYN Scan	S	16384	0	1436
5	Unicornscaan TCP Connect Scan				
6	Unicornscaan TCP NULL Scan	NULL	16384	0	1436

7	Unicornscaan TCP FIN Scan	F	16384	0	1436
8	Hping3 TCP SYN Scan	S	512	-	-
9	Hping3 TCP XMAS Scan	F+P+U	512	-	-
10	Hping3 TCP ACK Scan	A	512	-	-
11	Masscan scan	S	1024	-	-

Model Decision Tree tunggal menunjukkan kinerja paling unggul secara keseluruhan dibandingkan kedua skenario lainnya. Bahkan Decision Tree tunggal menghasilkan kesalahan pemblokiran paling rendah, sebanyak 231 paket terhadap 57498 paket normal. Kemampuan mempelajari pola statistik *traffic* memungkinkan model membedakan aktivitas normal dan serangan dengan lebih presisi. Pendekatan Decision Tree terbukti lebih adaptif daripada metode pencocokan *ruleset* yang diterapkan oleh Suricata.

Capaian *recall* sebesar 0.7245 pada Decision Tree mengindikasikan bahwa hampir sepertiga *traffic reconnaissance* gagal dideteksi. Hal ini membuktikan bahwa variasi teknik pemindaian pada Tabel 3.2 belum cukup representatif sebagai data latih. Model Decision Tree terbukti kurang mampu mengenali karakteristik menyeluruh dari teknik pemindaian pada Tabel 3.1. Teknik pemindaian yang berhasil lolos dari deteksi model Decision Tree disajikan dalam Tabel 5.7.

Tabel 5.7 Teknik pemindaian yang lolos dari model Decision Tree

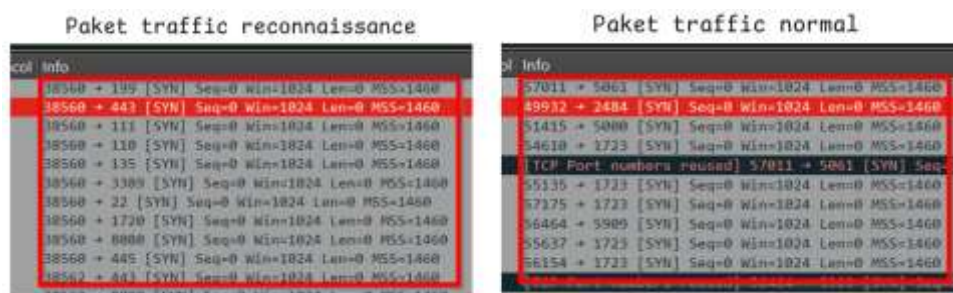
No	Teknik Pemindaian
1	Nmap TCP ACK Scan
2	Nmap TCP Window Scan
3	Unicornscaan TCP SYN Scan
4	Unicornscaan TCP Connect Scan
5	Hping3 TCP SYN Scan
6	Hping3 TCP ACK Scan
7	Masscan Scan

Kemampuan deteksi pada skenario *hybrid* IPS meningkat signifikan berkat kontribusi gabungan dari Suricata dan Decision Tree. Suricata berkontribusi memblokir 16144 paket *reconnaissance* yang sesuai dengan *ruleset*. Sementara itu, model Decision Tree berhasil menutup celah dengan mendeteksi 5028 serangan tambahan yang lolos dari Suricata. Hal ini membuktikan bahwa metode *hybrid* dapat menangkap paket *reconnaissance* yang tidak dikenali oleh pemindaian berbasis *ruleset*.

Peningkatan kemampuan deteksi pada skenario *hybrid* IPS mengalami *trade-off* dengan tingginya tingkat kesalahan blokir. Sebanyak 735 paket normal teridentifikasi salah sebagai *reconnaissance* selama proses pengujian. Komponen Suricata tercatat menyumbangkan angka kesalahan blokir yang

sangat signifikan hingga mencapai 713 paket. Sebaliknya, kontribusi kesalahan prediksi dari model Decision Tree sangat minim yakni 22 paket. Temuan ini konsisten dengan angka *false positive* yang muncul pada 2 pengujian sebelumnya. Skenario Decision Tree tunggal terbukti memiliki jumlah kesalahan deteksi yang jauh lebih rendah daripada Suricata tunggal. Akumulasi kesalahan dari satu komponen tersebut berdampak langsung pada penurunan kinerja sistem secara keseluruhan.

Tingginya angka *false positive* pada komponen Suricata disebabkan oleh paket pada *traffic* normal yang menyerupai salah satu teknik pemindaian. Beberapa paket normal memiliki *signature* yang sama dengan *ruleset* Nmap TCP SYN Scan. Gambar 5.6 memberikan visualisasi kemiripan beberapa paket normal dengan paket pada Nmap TCP SYN Scan. Hal ini dikarenakan keterbatasan fitur TCP yang bisa dideteksi oleh Suricata. Di sisi lain, model Decision Tree dapat melihat konteks fitur secara lebih luas sehingga lebih jarang melakukan kesalahan.

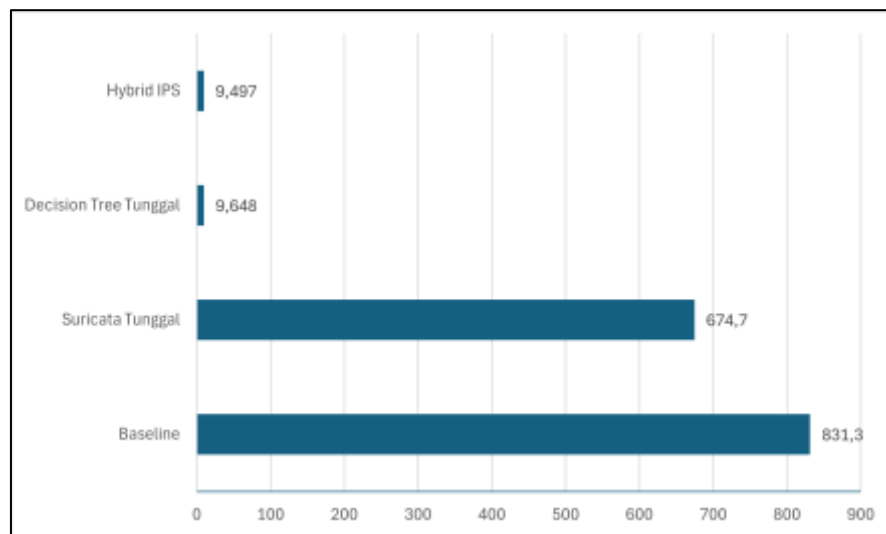


Gambar 5.6 Perbandingan paket *reconnaissance* dengan paket normal

Ketiga skenario pengujian akan memberikan dampak operasional yang berbeda jika diimplementasikan pada jaringan nyata. Implementasi Suricata tunggal menimbulkan risiko operasional berupa tingginya angka serangan yang lolos. Kondisi ini meningkatkan peluang celah bagi penyerang untuk mendapatkan informasi target secara leluasa. Skenario Decision Tree tunggal menawarkan keseimbangan performa paling optimal di dalam rangkaian pengujian ini. Angka kegagalan deteksi dan kesalahan blokir yang rendah sangat meminimalkan gangguan pada *traffic* normal. Sementara itu, penerapan *hybrid* IPS memberikan kompromi strategis melalui mekanisme pemblokiran dalam dua tahap. Pendekatan ini terbukti mampu meningkatkan pemblokiran ancaman *reconnaissance* dibandingkan Suricata tunggal. Namun, tingginya angka *false positive* dapat mengganggu ketersediaan layanan karena *traffic* normal yang salah terblokir. Oleh karena itu, implementasi model *hybrid* di lingkungan nyata membutuhkan kebijakan *whitelist*. Evaluasi berkala terhadap aturan Suricata yang memicu pemblokiran keliru juga sangat krusial untuk dilakukan. Pemantauan ini memastikan bahwa peningkatan proteksi keamanan tidak mengorbankan kualitas layanan secara keseluruhan.

5.2.2 Analisis *Throughput* Jaringan

Pengujian *baseline* menghasilkan *throughput* rata-rata 861.30 Mbps dengan variasi nilai yang sangat minim. Rentang nilai yang terukur stabil berada pada 847 hingga 876 Mbps. Stabilitas tinggi ini menunjukkan bahwa jaringan beroperasi optimal tanpa gangguan eksternal. Fluktuasi kecil yang muncul murni merupakan suatu fenomena yang standar dalam jaringan. Oleh karena itu, penurunan nilai pada skenario lain dapat dipastikan sebagai dampak langsung pemrosesan sistem. Gambar 5.7 memberikan visualisasi perbandingan rata-rata *throughput*.



Gambar 5.7 Perbandingan rata-rata *throughput* pada setiap skenario

Implementasi Suricata tunggal menyebabkan penurunan *throughput* menjadi rata-rata 674.70 Mbps. Nilai tersebut setara dengan 78.34% dari hasil rata-rata pada *baseline*. Penurunan sekitar 21.66% merupakan konsekuensi logis dari aktivitas inspeksi paket yang berjalan. Proses pencocokan *signature* pada setiap paket terhadap *ruleset* memerlukan waktu. Meskipun demikian, Suricata tunggal masih mampu mempertahankan aliran data ratusan Megabit per detik dengan lancar.

Rentang nilai *throughput* pada skenario Suricata tunggal justru terlihat lebih sempit dibandingkan skenario *baseline*. Nilai *throughput* bergerak sangat stabil di antara angka 663 hingga 684 Mbps. Hal ini mengindikasikan bahwa *overhead* pemrosesan telah mencapai titik jenuh yang konsisten. Sistem seolah terkunci pada kapasitas pemrosesan maksimal yang konstan sepanjang iterasi pengujian. Fluktuasi jaringan tidak lagi mendominasi variasi nilai karena batas atas ditentukan oleh kecepatan pemrosesan CPU.

Penerapan Decision Tree tunggal memicu penurunan *throughput* yang sangat drastis hingga tersisa 9.648 Mbps. Angka ini hanya merepresentasikan 1.12% dari kapasitas rata-rata pada *baseline*. Penurunan *throughput* terjadi karena model Decision Tree berjalan sepenuhnya pada *user space*.

Keterbatasan ini menyebabkan paket tidak dapat diproses dengan kecepatan yang memadai. Selain itu, proses ekstraksi fitur dan klasifikasi memerlukan waktu tambahan. Hal ini membuat paket harus tertahan dalam antrean Nfqueue hingga model memberikan hasil klasifikasi. Model Decision Tree pada penelitian Bertoli dkk., (2021) memerlukan waktu 3-4ms untuk melakukan klasifikasi. Meskipun waktu klasifikasi model pada penelitian ini tidak dihitung, penelitian Bertoli dkk., (2021) dapat menjadi acuan. Bahkan, di setiap iterasi pengujian mengalami kondisi *throughput* nol dengan rata-rata durasi 19.5 detik. Hal ini menandakan waktu pemrosesan yang dibutuhkan oleh model Decision Tree menghasilkan sebuah kondisi kemacetan dalam jaringan.

Skenario *hybrid* IPS menghasilkan kinerja *throughput* paling rendah dengan rata-rata 9.497 Mbps. Penurunan ini konsisten dengan penambahan beban komputasi dari gabungan *ruleset* dan model klasifikasi. Durasi kemacetan pada skenario ini meningkat hingga mencapai rata-rata 20.9 detik. Hal ini membuktikan bahwa kompleksitas komputasi berbanding lurus dengan risiko kemacetan *traffic*. Setiap penambahan tahapan analisis paket akan memangkas kapasitas pengiriman data secara signifikan.

Perbandingan hasil *throughput* pada tiap skenario memiliki implikasi operasional yang serius pada jaringan produksi. *Baseline* dan Suricata tunggal terbukti masih sanggup menangani lalu lintas data bervolume tinggi dengan baik. Sebaliknya, implementasi Decision Tree dan *hybrid* IPS berisiko menciptakan antrean paket yang sangat panjang. Penurunan *throughput* hingga di bawah 10 Mbps akan memicu latensi tinggi dan gangguan akses jaringan. Sistem yang seharusnya cepat berubah menjadi lambat akibat keterbatasan kemampuan komputasi dalam memproses paket.

BAB 6 PENUTUP

6.1 Kesimpulan

Penelitian ini berhasil melakukan implementasi *hybrid* IPS berbasis Suricata dan Decision Tree sebagai solusi memblokir *reconnaissance*. Berikut merupakan kesimpulan yang dapat diambil dari penelitian ini:

1. Sistem *hybrid* IPS diimplementasikan pada lingkungan penelitian yang dibuat dengan *virtual machine*. Sistem ini menggabungkan Suricata sebagai IPS berbasis *signature* dan Decision Tree sebagai IPS berbasis ML. Suricata bertugas sebagai lapisan pertama untuk memblokir *reconnaissance* sesuai *ruleset* yang telah dibuat. Paket yang lolos dari Suricata diteruskan ke komponen Decision Tree melalui antrean *Nfqueue*. Komponen Decision Tree bertugas melakukan klasifikasi terhadap teknik *reconnaissance zero-day*. Hasil klasifikasi dari model akan menentukan apakah paket tersebut harus diblokir atau diteruskan.
2. Pengujian kinerja dan *throughput* jaringan pada skenario IPS menghasilkan karakteristik yang berbeda:
 - a. Suricata tunggal menghasilkan *recall* paling rendah di antara skenario yang lain, dengan nilai 0.5338. Namun, Suricata unggul dengan rata-rata *throughput* paling tinggi yakni 674.70 Mbps.
 - b. Decision Tree tunggal menghasilkan keseluruhan nilai metrik evaluasi paling baik. Namun, penurunan rata-rata *throughput* Decision Tree tunggal sangat tajam hingga berada di angka 9.648 Mbps.
 - c. *Hybrid* IPS menghasilkan nilai metrik evaluasi mendekati Decision Tree tunggal, namun terjadi kenaikan *false positive* dikarenakan komponen Suricata. Rata-rata *throughput* yang dihasilkan sedikit turun dari Decision Tree dikarenakan pemrosesan oleh kedua komponen, dengan nilai 9.497 Mbps.

Pemilihan sistem akan bergantung dari kebutuhan setiap pengguna. Suricata cocok pada jaringan yang membutuhkan kecepatan tinggi, namun berpotensi meloloskan teknik *reconnaissance zero-day*. Di sisi lain, model Decision Tree dapat mendeteksi dan memblokir teknik *reconnaissance zero-day*. Namun, Decision Tree memiliki *trade-off* berupa penurunan *throughput* signifikan pada angka 9.648 Mbps. *Hybrid* IPS menghasilkan kinerja dan *throughput* mendekati Decision Tree dengan peningkatan *false positive*. Implementasi *hybrid* IPS perlu menimbang ulang potensi *false positive* yang dihasilkan oleh komponen Suricata.

6.2 Saran

Terdapat beberapa saran pengembangan lebih lanjut dari penelitian ini:

1. Diperlukan metrik pengukuran lain untuk menentukan *trade-off* dari setiap skenario. Metrik penggunaan sumber daya seperti CPU dan RAM, hingga metrik lain pada jaringan seperti latensi pemblokiran paket.
2. Pengujian sistem IPS terhadap serangan siber di luar lingkungan penelitian perlu dianalisis. Peneliti menyarankan sistem IPS dilatih menggunakan seluruh teknik pemindaian. Skenario pengujian dapat dilakukan dengan praktisi ahli yang mencoba melakukan serangan.
3. Implementasi model Decision Tree pada lapisan *kernel* dan mencoba pendekatan *multithreat* untuk mempercepat pemrosesan pada komponen model.
4. Mekanisme pemblokiran dapat dikembangkan lebih lanjut menjadi pemblokiran berbasis IP. Suricata dan Decision Tree bertugas sebagai deteksi serangan dan menghasilkan sebuah *blocklist* IP.

DAFTAR PUSTAKA

- Achleitner, S., La Porta, T.F., McDaniel, P., Sugrim, S., Krishnamurthy, S.V. and Chadha, R., 2017. Deceiving Network Reconnaissance Using SDN-Based Virtual Topologies. *IEEE Transactions on Network and Service Management*, 14(4), pp.1098–1112. <https://doi.org/10.1109/TNSM.2017.2724239>.
- Andress, J. and Winterfeld, S., 2014. *Cyber warfare: techniques, tactics and tools for security practitioners*. 2nd edition edn. Amsterdam Boston: Elsevier.
- Asad, H., Adhikari, S. and Gashi, I., 2024. A perspective–retrospective analysis of diversity in signature-based open-source network intrusion detection systems. *International Journal of Information Security*, 23(2), pp.1331–1346. <https://doi.org/10.1007/s10207-023-00794-9>.
- Asad, H. and Gashi, I., 2022. Dynamical analysis of diversity in rule-based open source network intrusion detection systems. *Empirical Software Engineering*, 27(1), p.4. <https://doi.org/10.1007/s10664-021-10046-w>.
- Bertoli, G.D.C., Pereira Junior, L.A., Sautome, O., Dos Santos, A.L., Verri, F.A.N., Marcondes, C.A.C., Barbieri, S., Rodrigues, M.S. and Parente De Oliveira, J.M., 2021. An End-to-End Framework for Machine Learning-Based Network Intrusion Detection System. *IEEE Access*, 9, pp.106790–106805. <https://doi.org/10.1109/ACCESS.2021.3101188>.
- Bilge, L. and Dumitraş, T., 2012. Before we knew it: an empirical study of zero-day attacks in the real world. In: *Proceedings of the 2012 ACM conference on Computer and communications security*. [online] CCS'12: the ACM Conference on Computer and Communications Security. Raleigh North Carolina USA: ACM. pp.833–844. <https://doi.org/10.1145/2382196.2382284>.
- Breiman, L., Friedman, J.H., Olshen, R.A. and Stone, C.J., 1984. *Classification And Regression Trees*. 1st edn. [online] Routledge. <https://doi.org/10.1201/9781315139470>.
- BSSN, 2024. LANSKAP KEAMANAN SIBER 2024.
- Coutinho, A.C. and Araújo, L.V.D., 2025. MICRA: A Modular Intelligent Cybersecurity Response Architecture with Machine Learning Integration. *Journal of Cybersecurity and Privacy*, 5(3), p.60. <https://doi.org/10.3390/jcp5030060>.
- Erney, T. and Chowdhury, M.M., 2022. A Survey of Intrusion Detection and Prevention Systems. In: *2022 IEEE World AI IoT Congress (AlloT)*. [online] 2022 IEEE World AI IoT Congress (AlloT). Seattle, WA, USA: IEEE. pp.578–584. <https://doi.org/10.1109/AlloT54504.2022.9817348>.

Fontugne, R., Borgnat, P., Abry, P. and Fukuda, K., 2010. MAWILab: combining diverse anomaly detectors for automated anomaly labeling and performance benchmarking. In: *Proceedings of the 6th International Conference*. [online] Co-NEXT '10: Conference on emerging Networking EXperiments and Technologies. Philadelphia Pennsylvania: ACM. pp.1–12. <https://doi.org/10.1145/1921168.1921179>.

Goyal, P. and Goyal, A., 2017. Comparative study of two most popular packet sniffing tools-Tcpdump and Wireshark. In: *2017 9th International Conference on Computational Intelligence and Communication Networks (CICN)*. [online] 2017 9th International Conference on Computational Intelligence and Communication Networks (CICN). Gurnee: IEEE. pp.77–81. <https://doi.org/10.1109/CICN.2017.8319360>.

Guo, Y., 2023. A review of Machine Learning-based zero-day attack detection: Challenges and future directions. *Computer Communications*, 198, pp.175–185. <https://doi.org/10.1016/j.comcom.2022.11.001>.

Huawei Technologies Co., Ltd., 2023. Virtualization Technology. In: *Cloud Computing Technology*. [online] Singapore: Springer Nature Singapore. pp.97–144. https://doi.org/10.1007/978-981-19-3026-3_3.

Hutchins, E., Cloppert, M. and Amin, R., 2011. Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains. *Leading Issues in Information Warfare & Security Research*, 1.

Kaur, S. and Singh, M., 2013. Automatic attack signature generation systems: A review. *IEEE Security & Privacy*, 11(6), pp.54–61. <https://doi.org/10.1109/MSP.2013.51>.

Kufel, J., Bargieł-Łączek, K., Kocot, S., Koźlik, M., Bartnikowska, W., Janik, M., Czogalik, Ł., Dudek, P., Magiera, M., Lis, A., Paszkiewicz, I., Nawrat, Z., Cebula, M. and Gruszczyńska, K., 2023. What Is Machine Learning, Artificial Neural Networks and Deep Learning?—Examples of Practical Applications in Medicine. *Diagnostics*, 13(15), p.2582. <https://doi.org/10.3390/diagnostics13152582>.

Langer, S.G. and French, T., 2011. Virtual Machine Performance Benchmarking. *Journal of Digital Imaging*, 24(5), pp.883–889. <https://doi.org/10.1007/s10278-010-9358-6>.

Liao, S., Zhou, C., Zhao, Y., Zhang, Z., Zhang, C., Gao, Y. and Zhong, G., 2020. A Comprehensive Detection Approach of Nmap: Principles, Rules and Experiments. In: *2020 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC)*. [online] 2020 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC). Chongqing, China: IEEE. pp.64–71. <https://doi.org/10.1109/CyberC49757.2020.00020>.

Mao, Z., Xu, Y. and Suarez, E., 2023. *Dataset Management Platform for Machine Learning*. <https://doi.org/10.48550/arXiv.2303.08301>.

Medina, V. and García, J.M., 2014. A survey of migration mechanisms of virtual machines. *ACM Computing Surveys*, 46(3), pp.1–33. <https://doi.org/10.1145/2492705>.

Netfilter Project, 2025a. *Man page of iptables-extensions*. [online] IP sets – Netfilter.org. Available at: <<https://ipset.netfilter.org/iptables-extensions.man.html>> [Accessed 17 September 2025].

Netfilter Project, 2025b. *The netfilter.org 'libnetfilter_queue' project*. [online] Netfilter Project. Available at: <https://www.netfilter.org/projects/libnetfilter_queue/index.html> [Accessed 17 September 2025].

Olivier, J. and Aldrich, C., 2021. Use of Decision Trees for the Development of Decision Support Systems for the Control of Grinding Circuits. *Minerals*, 11(6), p.595. <https://doi.org/10.3390/min11060595>.

Open Information Security Foundation (OISF), 2025. *14. Setting up IPS/inline for Linux — Suricata 6.0.14 documentation*. [online] Suricata Documentation. Available at: <<https://docs.suricata.io/en/suricata-6.0.14/setting-up-ipsinline-for-linux.html>> [Accessed 17 September 2025].

Ouiazane, S., Addou, M. and Barramou, F., 2022. A Suricata and Machine Learning Based Hybrid Network Intrusion Detection System. In: Y. Maleh, M. Alazab, N. Gherabi, L. Tawalbeh and A.A. Abd El-Latif, eds. *Advances in Information, Communication and Cybersecurity*, Lecture Notes in Networks and Systems. [online] Cham: Springer International Publishing. pp.474–485. https://doi.org/10.1007/978-3-030-91738-8_43.

Popek, G.J. and Goldberg, R.P., 1974. Formal requirements for virtualizable third generation architectures. *Communications of the ACM*, 17(7), pp.412–421. <https://doi.org/10.1145/361011.361073>.

Renear, A.H., Sacchi, S. and Wickett, K.M., 2010. Definitions of *dataset* in the scientific and technical literature. *Proceedings of the American Society for Information Science and Technology*, 47(1), pp.1–4. <https://doi.org/10.1002/meet.14504701240>.

Roy, S., Sharmin, N., Acosta, J.C., Kiekintveld, C. and Laszka, A., 2022. *Survey and Taxonomy of Adversarial Reconnaissance Techniques*. <https://doi.org/10.48550/arXiv.2105.04749>.

- Safavian, S.R. and Landgrebe, D., 1991. A survey of decision tree classifier methodology. *IEEE Transactions on Systems, Man, and Cybernetics*, 21(3), pp.660–674. <https://doi.org/10.1109/21.97458>.
- Sathyanarayanan, S. and Tantri, B.R., 2024. Confusion Matrix-Based Performance Evaluation Metrics. *African Journal of Biomedical Research*, pp.4023–4031. <https://doi.org/10.53555/AJBR.v27i4S.4345>.
- Schraven, J., Windmann, A. and Niggemann, O., 2025. *MAWIFlow Benchmark: Realistic Flow-Based Evaluation for Network Intrusion Detection*. <https://doi.org/10.48550/ARXIV.2506.17041>.
- Syamsuddin, I. and Barukab, O.M., 2022. SUKRY: Suricata IDS with Enhanced kNN Algorithm on Raspberry Pi for Classifying IoT Botnet Attacks. *Electronics*, 11(5), p.737. <https://doi.org/10.3390/electronics11050737>.
- Ting, K.M., 2011. Confusion Matrix. In: C. Sammut and G.I. Webb, eds. *Encyclopedia of Machine Learning*. [online] Boston, MA: Springer US. pp.209–209. https://doi.org/10.1007/978-0-387-30164-8_157.